Web Technologies

HTML5, CSS3, layout and responsive design

27 chapters | tested code with real browser output

Contents

PART A Web Basics

1. Internet, Web & Client-Server ... 2
2. URL, Domain, DNS, Hosting & SEO... 4
3. Web Standards & Application Typ... 6
4. Web Architecture & HTTP ... 8

PART B HTML5

5. HTML Basics & Document Structur... 12
6. Text, Links, Images & Media ... 15
7. Lists & Tables ... 18
8. Block vs Inline, div, span & Se... 21
9. HTML Forms ... 24

PART C CSS Fundamentals

10. Introduction to CSS ... 30
11. CSS Selectors ... 34
12. Cascade, Specificity & Inherit... 38
13. Values, Units, Colours, Fonts ... 42
14. Backgrounds & Borders ... 45
15. The Box Model, Sizes & Overflo... 47
16. Display, Visibility, Float & P... 51

PART D CSS Layout

17. Flexbox ... 57
18. CSS Grid ... 61
19. Responsive Design & Media Quer... 63

PART E CSS3 Features

20. CSS3 Visual Effects ... 69
21. Transform, Transition & Animat... 71
22. Web Fonts, Icon Fonts & CSS Va... 74
23. Compatibility: Prefixes, Fallb... 76

PART F Revision

24. Final Revision & Practice ... 78

Part A

Web Basics

1 Internet, Web & Client-Server

In one line: The Internet is the network; the Web is a service that runs on it; browsers (clients) ask servers for pages.

Internet vs World Wide Web

	Internet	World Wide Web (Web)
what	global system of interconnected computer networks	a collection of information (pages, files) you reach through the Internet
uses	TCP/IP protocol to link devices worldwide	HTTP / HTTPS, URLs, HTML
analogy	the roads	the shops and houses on those roads
also carries	email, file transfer, games, calls	web pages, web apps, videos

- **REMEMBER** The Web is only ONE service that runs on the Internet. Email and file transfer use the Internet but are not 'the Web'.

Client-Server model

- **Client** = device or program that REQUESTS data or services (Chrome, Firefox, Gmail, Outlook).
- **Server** = system that stores resources, manages data and RESPONDS to requests (web server Apache / Nginx, email server, database server).
- **Request-response cycle:** the client asks, the server answers - always in this order.
- **Centralised management:** data and services live on servers, so security and consistency are easier to control.
- A server handles **many clients at the same time** (concurrent request handling).

What happens when you type a URL?

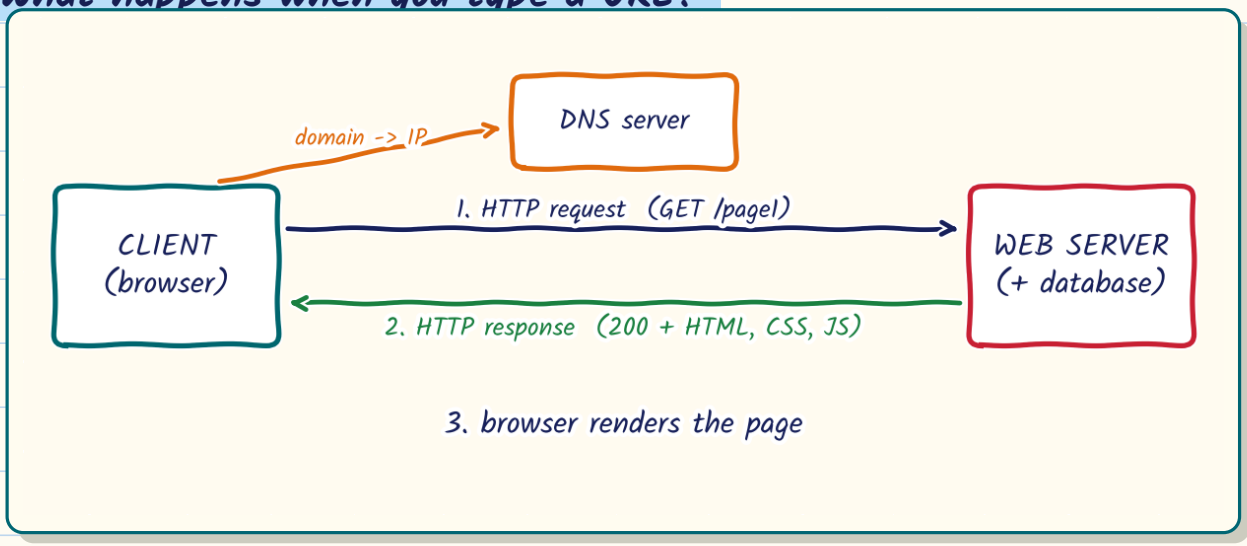


Figure: the request-response flow with a DNS lookup first.

Step	What happens
1	user types a URL, e.g. www.example.com
2	browser does a **DNS lookup**: domain name -> IP address
3	browser connects and sends an **HTTP / HTTPS request** to that IP
4	server responds with **HTML, CSS, JavaScript, images**
5	browser **renders** the page by processing these files

- **TIP** Memory trick: Type -> Translate (DNS) -> Talk (request) -> Tell back (response) -> Show (render).

2 URL, Domain, DNS, Hosting & SEO

In one line: The vocabulary of the Web: addresses, names, where sites live, and how they get found.

URI, URL and URN

- **URI** (Uniform Resource Identifier) = a string that identifies a resource by **location, name, or both**.
- A URI can be a **URL** (locator: where it is), a **URN** (name: what it is), or both.
- **URL** (Uniform Resource Locator) = the web address you type in the browser.

URN example: urn:daraz:product:123456789 (a NAME, no location)

URL example: https://www.daraz.pk/products/iphone-15-128gb

Part	Example	Meaning
protocol	https://	how to talk to the server (HTTP / HTTPS)
domain name	www.daraz.pk	human-readable address of the server
path	/products/iphone-15-128gb	which resource on that server

Protocol: HTTP vs HTTPS

	HTTP	HTTPS
full name	Hypertext Transfer Protocol	HTTP Secure
job	moves data between web server and client, defines the format of requests/responses	same, but data is ENCRYPTED
protects against	nothing - data can be read	eavesdropping and data tampering
use for	never for passwords or payments	every modern site, especially login / payment

Domain name and DNS

- A **domain name** is a human-readable address (www.example.com) that maps to an **IP address** (numbers).
- **DNS** (Domain Name System) = the 'phone book' of the Internet that translates the domain name into the IP address the browser needs.

Web hosting

- **Web hosting** = a service that gives storage, server resources and connectivity so your website is reachable on the Internet.

Type	How it works	Good for
Shared hosting	many websites on ONE server, sharing CPU, RAM, bandwidth	small / cheap sites
VPS (Virtual Private Server)	several virtual servers on one machine, each with its OWN dedicated resources, independent	growing sites
Dedicated hosting	an entire physical server for ONE user / organisation	big, high-traffic sites

SEO - Search Engine Optimisation

- SEO = improving a website so it appears **higher in search results** and gets more traffic.
- Main techniques: **content SEO** (useful, well-structured content), **keyword** research, **internal linking** (pages link to each other), **backlinks** (other sites link to you).

Web browser and web server

	Web browser (client)	Web server
job	gets files from a server and DISPLAYS the page	stores, processes and DELIVERS web pages
examples	Chrome, Firefox, Safari, Edge	Apache, Nginx, Microsoft IIS
runs on	desktop, laptop, tablet, phone	a computer (hardware + software) that speaks HTTP

3 Web Standards & Application Types

In one line: Standards make sites work the same everywhere; the type of app decides speed, cost and user experience.

Web standards bodies

- **Web standards** = official rules created by international organisations so websites work the same on all browsers and devices.

Body	Who / what	Looks after
W3C	World Wide Web Consortium, founded by **Tim Berners-Lee**	CSS, SVG, accessibility guidelines
WHATWG	formed by Apple, Mozilla, Google, Microsoft	the HTML and DOM **Living Standard**
ECMA (TC39)	standards organisation	**ECMAScript** - the standard behind JavaScript
IETF	Internet Engineering Task Force	core protocols: HTTP, TCP, URIs, MIME

Types of web applications

Type	How it works	Pros	Cons	Ideal for
Static	pre-built HTML / CSS / JS files; server sends the same file to everyone	very fast, secure, cheap	hard to update live; no user accounts without APIs	portfolios, docs, landing pages
Dynamic	server builds the page on request: fetches database data + HTML template	personalised, logins, real-time data	slower, complex to build and scale	e-commerce, banking, CMS (WordPress)
MPA (multi-page	every click loads a NEW page from the server (screen blinks)	great SEO (own URL per page)	page reloads hurt user experience	Amazon, eBay, news portals

Type	How it works	Pros	Cons	Ideal for
SPA (single-pag	one HTML + big JS file; JS fetches data and rewrites the page without reload	fluid, app-like, fast after first load	slow first load; harder SEO	Gmail, Facebook, X, dashboards
PWA (progressive	website that behaves like a native app using Service Workers to cache files	installable, works OFFLINE, push notifications, no app store	limited native hardware access	mobile-first and productivity tools

- **FACT** A PWA is not a replacement for SPA / MPA - it is an ENHANCEMENT layer built on top of them.

Quick way to pick

If you need...	Choose
cheap, fast, same content for all	Static
logins and live data	Dynamic
best SEO with many pages	MPA
smooth app-like screens	SPA
installable + offline on phones	PWA

4 Web Architecture & HTTP

In one line: Apps are split into layers (tiers); HTTP is the language browsers and servers use to talk.

The three layers of a web application

Layer	Also called	Job	Example (online shop)
Presentation	UI layer	what the user SEES and interacts with; takes input, shows output	pages, buttons, forms, menus, images
Application	business-logic layer	models + logic; sits BETWEEN the UI and the database	calculate cart total, check login, apply discount
Data	database layer	stores and retrieves data	products, users, orders

1-tier, 2-tier, 3-tier (N-tier)

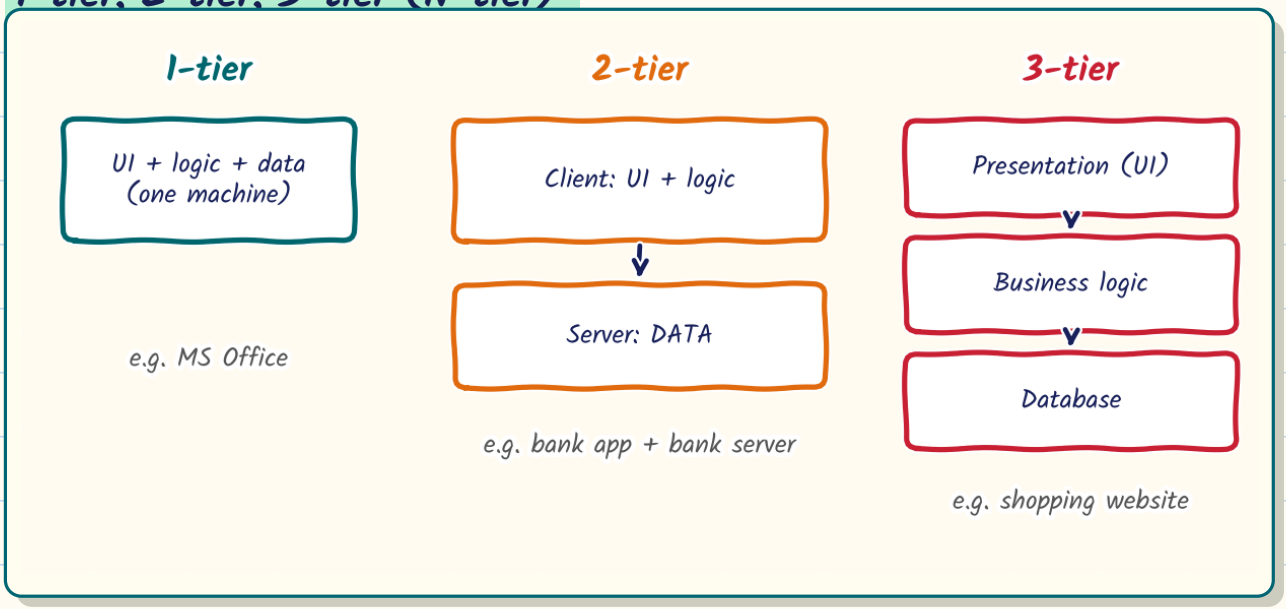


Figure: where the layers live in each architecture.

Architecture	Where the layers are	Pros / cons	Example

Architecture	Where the layers are	Pros / cons	Example
1-tier (single-tier, stand-alone)	UI, logic and data all in ONE package on ONE machine (client and server are the same machine)	+ cheap, simple, easy to maintain - cannot share data, not suitable for web apps	MS Office
2-tier (client-server	client handles Presentation + Application; server handles the Database	client sends request, server processes and returns data	bank app: balance, history, transfer
3-tier	Presentation, Business and Database on SEPARATE machines (one server for logic, one for database)	more secure, scalable, easier to update one layer	big web applications
N-tier	more than three separate layers / servers	even more flexible and scalable	large enterprise systems

HTTP - HyperText Transfer Protocol

- Foundation of data communication on the Web: exchanges text, images, videos, links between browser and server.
- Works on a **request-response** model.
- **Stateless:** every request is INDEPENDENT - the server does not remember previous requests.
- HTTP uses a **TCP** connection (TCP sets up a reliable link between the two machines).

HTTP step by step (example: http://www.example.com/page1)

Who	Step
Client	types the URL; browser builds a request with the **method** (GET), the **domain** (www.example.com) and the **path** (/page1)
Client	opens a **TCP connection** to the server and sends the request
Server	receives and parses the request, finds /page1 (a file or generated content)

Who	Step
Server	builds the response: **status code** (200 OK / 404 Not Found), **content type** (text/html) and the content (the HTML)
Client	receives the response, renders the HTML and shows the page
Both	the TCP connection is closed - the HTTP transaction is complete

```
REQUEST (client -> server)
GET /page1 HTTP/1.1
Host: www.example.com
Accept: text/html
```

```
RESPONSE (server -> client)
HTTP/1.1 200 OK
Content-Type: text/html

<html> ...page1 content... </html>
```

Common HTTP methods and status codes

Method	Meaning
GET	ask for a resource; form data goes in the URL
POST	send data (login, file upload) in the request BODY

Code	Meaning
200	OK - success
301	moved permanently (redirect)
403	forbidden
404	Not Found
500	server error

Part B

HTML5

5 HTML Basics & Document Structure

In one line: HTML gives a web page its structure and meaning: headings, paragraphs, links, images, tables, forms.

What is HTML?

- **HTML = HyperText Markup Language** - the standard language for structuring web content.
- It is a **markup language, NOT a programming language**: text + special tags that describe what the text IS.
- Markup adds meaning: <h1> means heading, <p> means paragraph.
- Why we need it: gives structure, helps browsers understand content, supports **accessibility** (screen readers), helps **search engines**, connects documents and embeds resources.

Technology	Job	Analogy
HTML	structure and content	skeleton
CSS	presentation and visual design	appearance (clothes)
JavaScript	behaviour and interactivity	behaviour (muscles)

The basic HTML document

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My Page</title>
</head>
<body>
  <h1>Welcome</h1>
  <p>My first webpage.</p>
</body>
</html>
```

browser output

Welcome

My first webpage.

Part	What it is
<!DOCTYPE html>	a declaration (NOT an element) at the very top; makes the browser use standards mode; HTML5 uses this short form
<html lang="en">	root element - everything except DOCTYPE is inside it; lang tells the language (helps accessibility + search engines)
<head>	metadata, mostly NOT visible: <title>, <meta>, <link>, <style>, <script>; e.g. <meta charset="UTF-8">
<body>	the VISIBLE content: headings, paragraphs, images, links, lists, tables, forms, media

```
html
 |-- head  -> metadata, title
 '-- body  -> visible content        (the document is a TREE)
```

Elements, tags, void elements, attributes

Term	Meaning	Example
tag	markup inside angle brackets	
opening / closing tag	start and end of an element	...
element	opening tag + content + closing tag	Hello World
void element	no content, NO closing tag	 <hr> <img> <meta> <link> <input>
attribute	extra information, written name="value"	<a href="https://example.com">

Attribute	Does
id	identifies ONE element (unique)

Attribute	Does
class	groups elements
style	inline CSS
title	extra info (tooltip)
href	link destination
src	location of a resource (image, script)
alt	alternative text for an image
width height	size

Nesting, comments, validation

Correct: This is important.

Incorrect: Hello (tags cross each other!)

```
<!-- This comment is NOT shown on the page -->
<p>This is <strong>important</strong> text.</p>
```

browser output

This is **important** text.

- Elements must be **properly nested** - like a parent-child tree. Close the inner tag first.
- Comments <!-- ... --> are notes for developers; useful for documentation or temporarily disabling code.
- **Validation** checks that HTML follows the standard (finds wrong nesting, invalid attributes, missing info). Benefits: better code quality, standards compliance, cross-browser reliability.
- HTML is NOT compiled and is very forgiving, so errors may go unnoticed - use the **W3C Validator** (validator.w3.org).

6 Text, Links, Images & Media

In one line: *The everyday building blocks of a page.*

Headings and paragraphs

```
<h1>Main heading</h1>
<h2>Sub heading</h2>
<h3>Smaller heading</h3>
<p>This is a paragraph of text.</p>
```

browser output

Main heading

Sub heading

Smaller heading

This is a paragraph of text.

- Six levels: <h1> (most important) to <h6>. Follow the hierarchy h1 -> h2 -> h3.
- Do NOT choose a heading just because it looks bigger - CSS controls looks.

Text formatting

Tag	Meaning	Tag	Meaning
	strong importance	<del>	deleted text
	emphasis	<ins>	inserted text
<mark>	highlighted	<sub>	subscript
<small>	smaller side text	<sup>	superscript

```
<p>Water is H<sub>2</sub>O and 2<sup>3</sup> = 8.</p>
<p><strong>Strong</strong>, <em>emphasis</em>, <mark>marked</mark>,
<small>small</small>, <del>deleted</del>, <ins>inserted</ins></p>
```

```
<pre>Preformatted   text
  keeps spaces</pre>
```

browser output

Water is H_2O and $2^3 = 8$.

Strong, *emphasis*, marked, small, ~~deleted~~, inserted

```
Preformatted    text
   keeps spaces
```

- **CAREFUL** <b> <i> <u> <font> are old presentation tags (deprecated). Use semantic tags + CSS instead.

Hyperlinks and URLs

```
<a href="https://www.google.com">Go to Google</a><br>
<a href="about.html">About (relative link)</a><br>
<a href="#section2">Jump to a section</a><br>
<a href="mailto:me@example.com">Email me</a>
```

browser output

Go to Google
About (relative link)
Jump to a section
Email me

Absolute URL	Relative URL
full address https://www.comsats.edu.pk	path from the CURRENT page, e.g. about.html
use for external websites	use for pages / files inside your own site

Images

```
<img src="campus.jpg" alt="University campus" width="300" height="150">
```

browser output

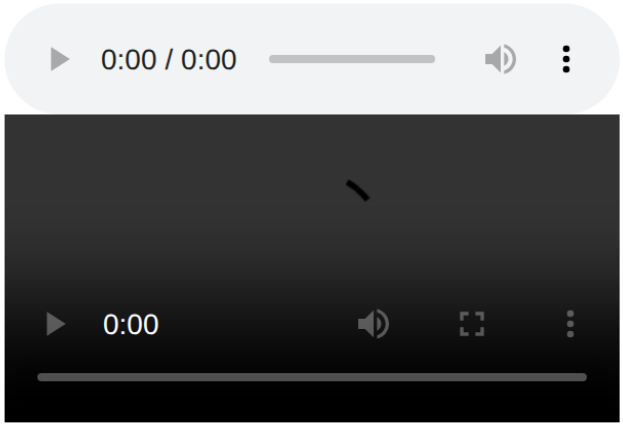


- src = image location, alt = text shown if the image fails and read by screen readers (**meaningful alt = accessibility**).
- width / height set the size; <img> is a void element.

Audio, video and iframe

```
<audio controls>
  <source src="song.mp3" type="audio/mpeg">
</audio>
<video controls width="300">
  <source src="clip.mp4" type="video/mp4">
</video>
```

browser output



- HTML5 has **native** media: <audio> and <video>. The controls attribute shows play / pause / volume.
- <source> gives the file and its MIME type (Multipurpose Internet Mail Extension) - the browser picks the first it can play.

```
<iframe src="https://www.youtube.com/embed/VIDEO_ID"
        title="Video" width="400" height="225"></iframe>
```

- <iframe> embeds external content (YouTube videos, maps). Give a width, height and a descriptive title.

7 Lists & Tables

In one line: Lists for menus and steps, tables for data in rows and columns.

Ordered and unordered lists

```
<ol type="1">
  <li>Apple</li><li>Orange</li><li>Grapefruit</li>
</ol>
<ul type="square">
  <li>HTML</li><li>CSS</li>
</ul>
```

browser output

1. Apple
2. Orange
3. Grapefruit

- HTML
- CSS

Tag	Type attribute values
ordered (numbers)	1, A, a, I, i
unordered (bullets)	disc, circle, square

- Both use <li> items. Lists are great for menus, steps, features, collections.
- Lists can be **nested** (a <ul> inside an <li>).

```
<dl>
  <dt>HTML</dt><dd>A markup language ...</dd>
  <dt>CSS</dt><dd>Language used to style ...</dd>
```

```
</dl>
```

browser output

HTML
    A markup language ...
CSS
    Language used to style ...

- **Definition list** <dl>: term in <dt>, definition in <dd>. No bullets; the definition is indented.

Basic table

```
<style>td,th{border:1px solid #333;padding:6px}
table{border-collapse:collapse}</style>
<table>
  <tr><th>Full Name</th><th>Mark</th></tr>
  <tr><td>Ali Khan</td><td>Very good (5)</td></tr>
  <tr><td>Sara Ahmed</td><td>Excellent (6)</td></tr>
</table>
```

browser output

Full Name	Mark
Ali Khan	Very good (5)
Sara Ahmed	Excellent (6)

Tag	Meaning
<table>	the table
<tr>	table row
<th>	header cell (bold, centred) - semantic: column names
<td>	data cell

- **CAREFUL** Use tables for TABULAR DATA only, never for page layout - use CSS (flexbox / grid) for layout.

Complete table: thead, tbody, tfoot, colgroup

```
<style>td,th{border:1px solid #333;padding:6px}
table{border-collapse:collapse}</style>
<table>
 <colgroup><col style="width:120px"><col></colgroup>
 <thead><tr><th>Column 1</th><th>Column 2</th></tr></thead>
 <tbody>
  <tr><td>Cell 1.1</td><td>Cell 1.2</td></tr>
  <tr><td>Cell 2.1</td><td>Cell 2.2</td></tr>
 </tbody>
 <tfoot><tr><td>Footer 1</td><td>Footer 2</td></tr></tfoot>
</table>
```

browser output

Column 1	Column 2
Cell 1.1	Cell 1.2
Cell 2.1	Cell 2.2
Footer 1	Footer 2

- <thead> header (uses <th>), <tbody> the data rows, <tfoot> footer. <colgroup> / <col> set column widths.

Spacing, nesting and spans

Attribute	Defines
cellpadding	empty space INSIDE a cell (around its content)
cellspacing	empty space BETWEEN cells
colspan="2"	cell takes 2 columns
rowspan="2"	cell takes 2 rows

```
<style>td{border:1px solid #333;padding:6px}
table{border-collapse:collapse}</style>
<table>
 <tr><td>Cell[1,1]</td><td colspan="2">Cell[2,1]</td></tr>
 <tr><td>Cell[1,2]</td><td rowspan="2">Cell[2,2]</td><td>Cell[3,2]</td></tr>
```

```
<tr><td>Cell[1,3]</td><td>Cell[2,3]</td></tr>
</table>
```

browser output

<table>
<tr><td>Cell[1,1]</td><td colspan="2">Cell[2,1]</td></tr>
<tr><td>Cell[1,2]</td><td rowspan="2">Cell[2,2]</td><td>Cell[3,2]</td></tr>
<tr><td>Cell[1,3]</td><td>Cell[2,3]</td></tr>
</table>

```
<style>td{border:1px solid #333;padding:6px}</style>
<table><tr><td>Contact:</td><td>
 <table><tr><td>First Name</td><td>Last Name</td></tr></table>
</td></tr></table>
```

browser output

<table>
<tr><td>Contact:</td><td><table><tr><td>First Name</td><td>Last Name</td></tr></table></td></tr>
</table>

- A <td> can contain another table (**nested table**).

8 Block vs Inline, div, span & Semantic HTM

In one line: *How elements flow on the page, and how HTML5 gives the page meaningful sections.*

Block-level vs inline elements

	Block-level	Inline
new line?	ALWAYS starts on a new line	does NOT start a new line
width	takes the FULL available width	only as much width as needed
spacing	browser adds margin before / after	none
examples	<p> <h1> <table>	<a> <img>

```
<style>.b{background:#ffd9d9;border:1px solid red}
.i{background:#d9ecff;border:1px solid blue}</style>
<p class="b">Block A (p)</p>
```

```
<div class="b">Block B (div)</div>
<span class="i">inline 1</span> <span class="i">inline 2</span>
<a class="i" href="#">link</a>
```

browser output

Block A (p)

Block B (div)
inline 1 inline 2 link

and - generic containers

```
<div style="font-size:24px; color:red">DIV example</div>
<p>This one is <span style="color:red; font-weight:bold">only a test.</span></p>
```

browser output

DIV example

This one is **only a test.**

block element	inline element
logical division of the page	styles a PART of a text
creates a separate area (new line)	does not create a separate area
used with CSS for layout	used with CSS for small pieces

HTML5 - what is new?

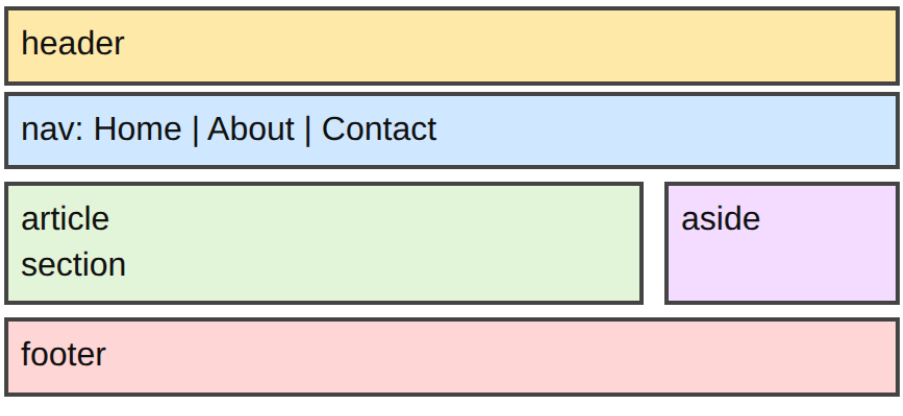
- Richer **semantic** structure, native **audio / video**, better **forms** (new input types), and APIs for modern web apps.
- Modern Chrome, Edge, Firefox and Safari support most commonly used features; newer / special APIs vary - check compatibility.

Semantic elements

Element	Meaning
<header>	introductory / header content
<nav>	navigation links
<main>	main content (once per page)
<section>	a thematic section
<article>	self-contained content (blog post, news item)
<aside>	related content / sidebar
<footer>	footer information

```
<style>*{box-sizing:border-box}
header,nav,main,aside,footer{border:2px solid #444;padding:6px;margin:3px}
header{background:#ffe9a8}nav{background:#cfe8ff}main{background:#e3f5d8}
aside{background:#f3dcff}footer{background:#ffd6d6}
.row{display:flex;gap:4px}.row main{flex:3}.row aside{flex:1}</style>
<header>header</header>
<nav>nav: Home | About | Contact</nav>
<div class="row"><main><article>article</article> <section>section</section></main>
<aside>aside</aside></div>
<footer>footer</footer>
```

browser output



- Benefits: **accessibility** (screen readers understand the structure), **maintainability**, **SEO**.
- Semantic > : <nav> says 'navigation', says nothing.

- **REMEMBER** Accessibility = making sites usable by people with different abilities and assistive technology. Semantic HTML is the easiest way to get it.

9 HTML Forms

In one line: Forms collect data from visitors and send it to a server.

The <form> element

```
<form id="myForm" method="post" action="path/to/script.php">
  <input type="text" name="user">
  <button type="submit">Send</button>
</form>
```

browser output



Attribute	Meaning
action	WHERE the form data is sent (a script / URL)
method	HOW it is sent: GET or POST

GET	POST
data appended to the URL (visible in address bar)	data sent in the BODY of the request (not in the URL)
for non-private data: search queries	for sensitive or large data: logins, file uploads

Common input attributes

Attribute	Does
type	what kind of input to show
name	the field's name / key when sending data
id	unique id (used by <label for>)
value	the actual / default value

Attribute	Does
placeholder	temporary hint text

```
<label for="username">Username:</label>
<input type="text" id="username" name="username"
    placeholder="Enter username" required pattern="[A-Za-z]{3,}">
```

browser output

Username: Enter username

Client-side validation attributes

Attribute	Meaning
required	must be filled before submit
min / max	smallest / largest allowed value
pattern	a regular expression the input must match
placeholder autofocus multiple	hint text, focus on load, allow many values

Form fields

```
<p>Text: <input type="text" name="FirstName" value="text field"></p>
<p>Password: <input type="password" name="pass" value="secret"></p>
<p><textarea name="Comments" rows="2" cols="30">multi-line text</textarea></p>
<input type="hidden" name="Account" value="not shown to user">
```

browser output

Text: text field

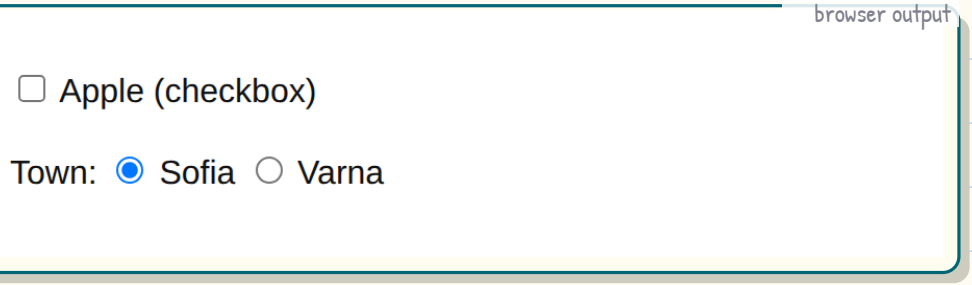
Password: ••••••

multi-line text

- type="hidden" holds data the user does NOT see. password hides characters with dots.

```
<p><input type="checkbox" name="fruit" value="apple"> Apple (checkbox)</p>
<p>Town: <input type="radio" name="town" value="Sofia" checked> Sofia
  <input type="radio" name="town" value="Varna"> Varna</p>
```

browser output

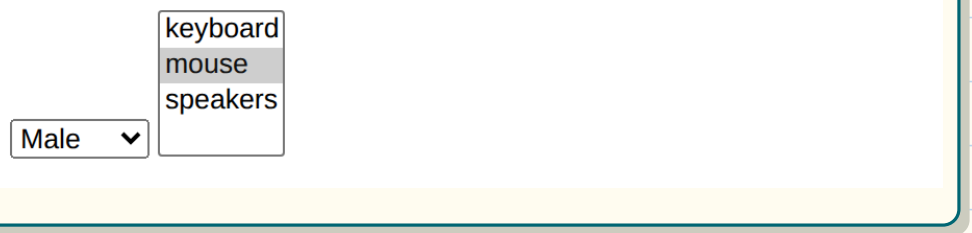


- **Checkbox:** *many can be ticked; its value is sent only if checked.*
- **Radio buttons** *with the SAME name form a group - only ONE can be selected.*

Drop-downs and buttons

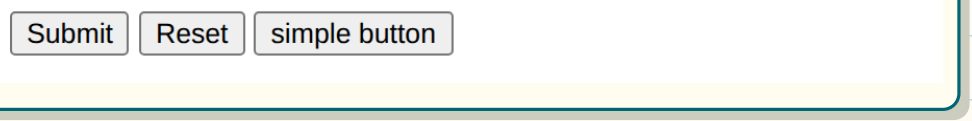
```
<select name="gender">
  <option value="1" selected>Male</option>
  <option value="2">Female</option>
  <option value="3">Other</option>
</select>
<select name="products" multiple>
  <option>keyboard</option><option selected>mouse</option><option>speakers</option>
</select>
```

browser output




```
<button type="submit">Submit</button>
<button type="reset">Reset</button>
<input type="button" value="simple button">
```

browser output



Button	Does
type="submit"	sends the form
type="reset"	clears the form
type="button"	no default action (used with JavaScript)

Button	Does
type="image"	acts like submit but shows an image

HTML5 input types

```
<p>Email: <input type="email" placeholder="you@mail.com"></p>
<p>Date: <input type="date"></p>
<p>Number: <input type="number" min="1" max="10" value="5"></p>
```

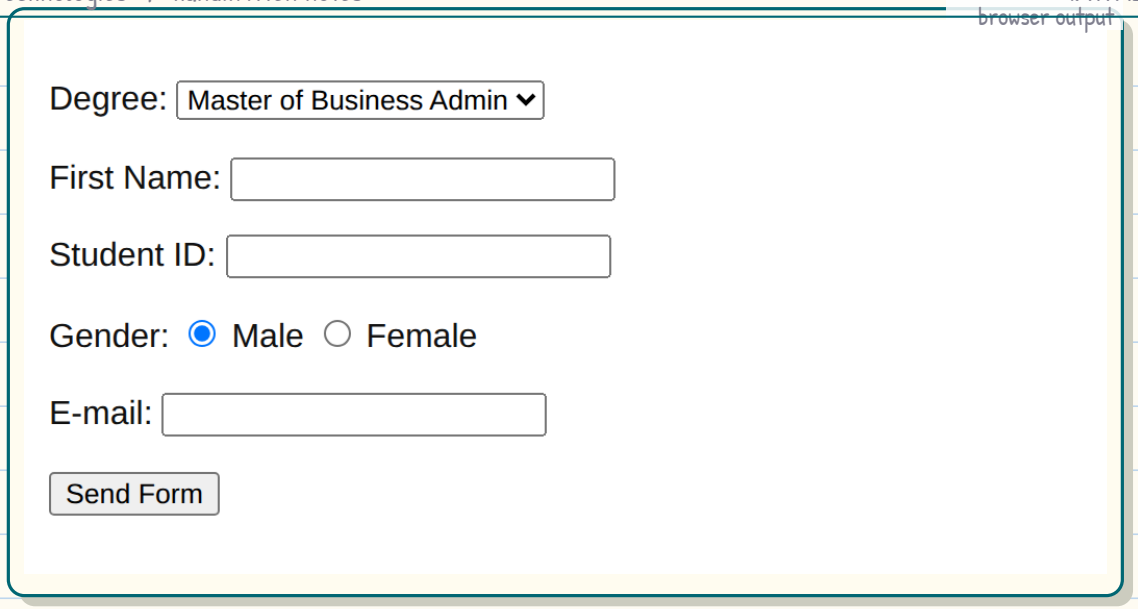
browser output



Complete form example

```
<form method="POST" action="apply-now.php">
 <p>Degree: <select name="degree">
  <option value="BS">Bachelor of Science</option>
  <option value="MBA" selected>Master of Business Admin</option></select></p>
 <p>First Name: <input type="text" name="firstname"></p>
 <p>Student ID: <input type="password" name="studentid"></p>
 <p>Gender: <input name="gender" type="radio" value="Male" checked> Male
   <input name="gender" type="radio" value="Female"> Female</p>
 <p>E-mail: <input type="email" name="email"></p>
 <p><input type="submit" value="Send Form"></p>
</form>
```

browser output



- **TIP** Always connect a label to its input: <label for="x"> + <input id="x">. Clicking the label then focuses the field and screen readers announce it.

Part C

CSS Fundamentals

10 Introduction to CSS

In one line: HTML says WHAT is on the page; CSS says HOW it looks.

Content vs appearance

- **HTML** describes the content. **CSS** (Cascading Style Sheets) describes how that content looks: colour, text, fonts, spacing, borders, layout.
- Keeping CSS separate from HTML makes changes easier (change one file -> the whole site changes).

Anatomy of a CSS rule

```
selector { property : value ; }
hr       { width    : 50%   ; }
```

Part	Meaning
selector	WHICH HTML elements to match
property	WHAT part of the appearance to change
value	the setting to use
declaration	property: value; (ends with a semicolon)

Syntax and style rules

- Put the <style> element in the <head>.
- Short rule: selector { property: value; }. A **semicolon after every declaration**.
- Several selectors: separate them with a **comma** and a space. Longer rules: **one declaration per line**.
- Comments: /* this is a CSS comment */.

```
<style>
  p {
    color: blue;
    text-align: center;
  }
     /* h2 and p share one rule below */
```

```
  h2, p { font-style: italic; }
</style>
<h2>Heading</h2>
<p>Blue, centred, italic.</p>
```

browser output

Heading

Blue, centred, italic.

Three ways to add CSS

Way	Where	Notes
Inline	style attribute on ONE element	no selector needed; overrides others; mixes content with looks - avoid; inline styles are being discouraged by W3C
Embedded / internal	<style> tag in <head>	applies to the whole page
External	separate .css file linked with <link rel="stylesheet" href="..."> in <head>	one file styles the WHOLE site; browser caches it; **highly recommended**

```
<!-- 1. INLINE -->
<p style="font-size:20pt; color:#0000FF">Even more text</p>
```

browser output

Even more text

```
<!-- 2. INTERNAL (in <head>) -->
<style> p { color: green; } </style>
```

```
<!-- 3. EXTERNAL (in <head>) + a separate file styles.css -->
<link rel="stylesheet" href="styles.css">
```

```
/* styles.css - only CSS rules and comments, no HTML tags */
a       { text-decoration: none }
a:hover  { text-decoration: underline; color: red }
ul      { margin-left: 2cm }
```

- **FACT** rel says what the linked file IS to the page (here: a stylesheet). The <link> tag can only be inside <head>.

Priority of the places (as in the lecture)

Order	Source
1 (highest)	style attribute (inline)
2	style element (internal)
3	external CSS file
4	user browser settings
5 (lowest)	browser default styles

- **REMEMBER** In real browsers, conflicts are settled by SPECIFICITY first, then by ORDER (the later rule wins). Inline beats id beats class beats element - see chapter 12.

Why CSS? What to avoid

Benefits	Avoid
more powerful formatting than presentation tags	<font>, , <b> for looks
pages load faster (CSS files are cached)	inline CSS everywhere
more accessible (rules per media / device)	very complex rule-sets (slow rendering)
easier to maintain and update	repeating the same style in many places

Example: Tree poem (3 rules)

```
<style>
  * { text-align: center; }
  hr { width: 50%; }
  h2, p { font-style: italic; color: blue; }
</style>
<h2>Trees</h2>
<hr>
<p>I think that I shall never see<br>A poem lovely as a tree.</p>
```

browser output

Trees

I think that I shall never see
A poem lovely as a tree.

- * centres ALL text, hr is 50% of its container, h2, p are italic and blue - one rule can format several element types.

11 CSS Selectors

In one line: A selector tells the browser WHICH elements a rule applies to.

Type, universal and grouping selectors

```
<style>
  hr { width: 50%; }            /* type selector */
  * { font-family: Arial; }     /* universal: every element */
  h2, p { color: darkgreen; }   /* grouping with a comma */
</style>
<h2>Title</h2><hr><p>Paragraph text</p>
```

browser output

Title

Paragraph text

Selector	Matches
p	all <p> elements (type selector = HTML element name)
*	EVERY element
h2, p	h2 elements AND p elements

Class selectors (.)

- Starts with a **dot**: .red matches elements with class="red".
- A class does NOT have to be unique - many elements can share it; an element can have several classes (class="a b").
- Name classes by purpose, not looks.

```
<style>
  .red { background-color: tomato; }
  q.blue { background-color: skyblue; }     /* only <q> with class blue */
  h1.blue { color: blue; }
  *.big-warning { font-size: x-large; color: red; }
```

```
</style>
<q class="red">A quote</q> <q class="blue">another</q>
<h1 class="blue">Heading</h1>
<p class="big-warning">Big warning!</p>
```

browser output

"A quote" "another"

Heading

Big warning!

- q.blue = ONLY q elements with class blue (more specific than .blue).
- *.big-warning = any element with that class (same as .big-warning).

ID selectors (#)

```
<style>
  #lizards-lounge { color: green; }
  .warning { color: red; }
</style>
<h3 id="lizards-lounge">Lizards Lounge</h3>
<p class="warning">Warning text</p>
```

browser output

Lizards Lounge

Warning text

	Class	ID
symbol	.name	#name
how many elements	MANY can share it	ONE unique element per page
use for	groups / repeated styles	one special element

Combinators - relationship between selectors

Combinator	Symbol	Selects	Example
descendant	space	ALL matching elements INSIDE (children, grandchildren...)	div p
child	>	only DIRECT children	div > p
next sibling	+	the FIRST element right after, same parent	div + p
subsequent sibling	~	ALL later siblings, same parent	div ~ p

```
<style>
  div p { color: blue; }              /* every p inside div */
  div > p { font-weight: bold; }      /* only direct children */
  h2 + p { background: yellow; }      /* p right after h2 */
  h2 ~ span { color: red; }           /* all spans after h2 */
</style>
<div><p>direct child</p><section><p>grandchild</p></section></div>
<h2>Title</h2><p>next sibling p</p><span>a span</span>
```

browser output

direct child

grandchild

Title

next sibling p

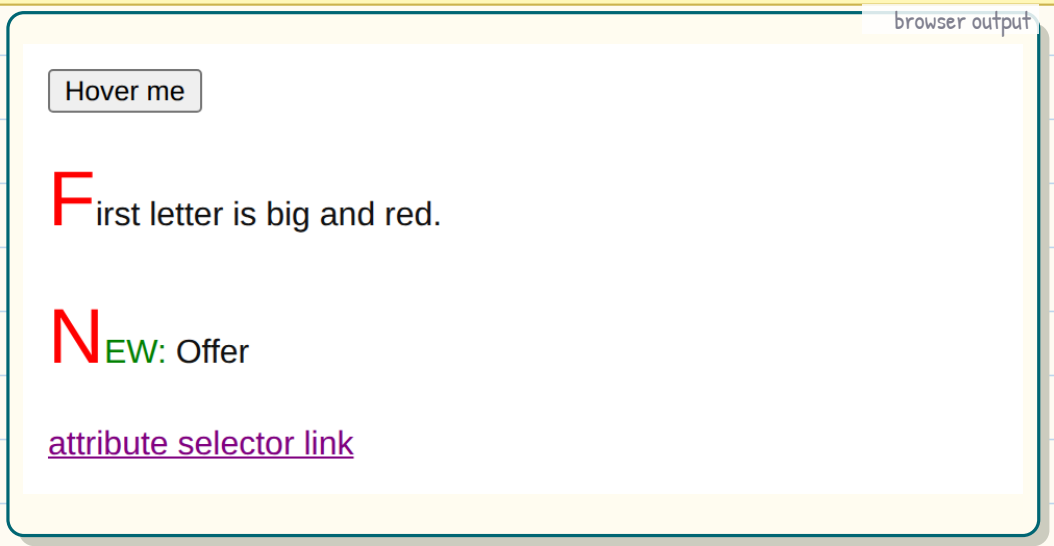
a span

Pseudo-classes (:) and pseudo-elements (::)

- **Pseudo-class** = a special STATE of an element; **ONE colon**: :hover (mouse over), :visited / :link (links), :focus (field selected), :first-child.
- **Pseudo-element** = a PART of an element; **TWO colons**: ::first-letter, ::first-line, ::before, ::after (insert content).

```
<style>
  .submit-btn:hover { background-color: aquamarine; }
  p::first-letter { font-size: 28pt; color: #ff0000; font-variant: small-caps; }
  .tag::before { content: "NEW: "; color: green; }
  a[href] { color: purple; }     /* attribute selector */
</style>
<button class="submit-btn">Hover me</button>
<p>First letter is big and red.</p>
<p class="tag">Offer</p>
<a href="#">attribute selector link</a>
```

browser output



span and div as CSS containers

```
<style>
  .orange { color: darkorange; }
  .white { color: white; }
  .orange-background { background-color: orange; }
</style>
<div class="orange">Text inside a div block.</div>
Price: <span class="white orange-background">$10</span>
```

browser output

Text inside a div block.
Price: $10

- span for text INSIDE a line, div for a BLOCK. By themselves they add no meaning - only containers for CSS.

12 Cascade, Specificity & Inheritance

In one line: When rules conflict, the browser decides the winner using three ideas.

Stylesheet conflicts

- As CSS grows, rules will conflict. Browsers resolve it with **Cascading**, **Specificity** and **Inheritance**.

1. Cascading - order matters

```
<style>
  a { color: purple; }
  a { color: green; }     /* same selector, later wins */
</style>
<a href="#">Which colour am I?</a>
```

browser output

Which colour am I?

- Both rules target all <a>; the browser can't do both -> the LATER rule (green) wins.

2. Specificity - a score for each selector

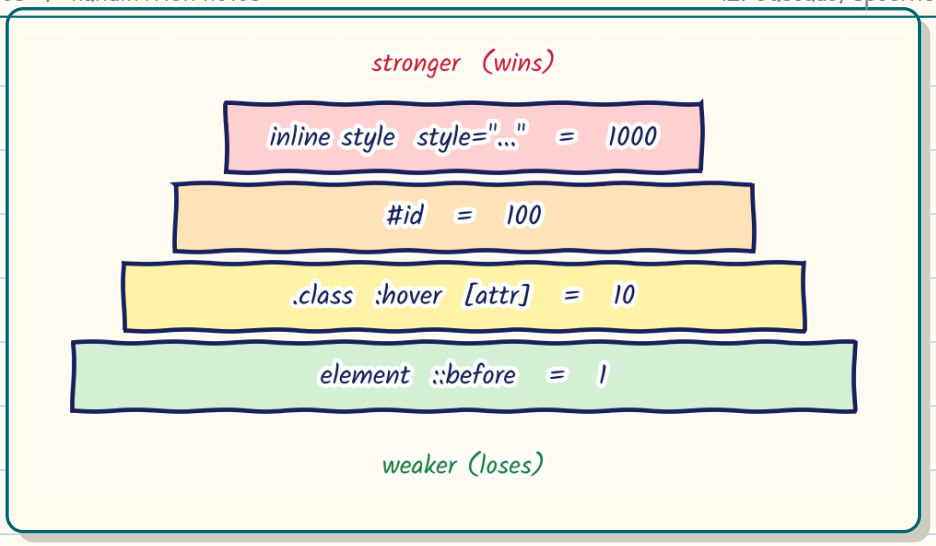


Figure: the specificity ladder. Higher = wins.

Selector	Value	Calculation
p	1	1
p .test	11	1 + 10
p #demo	101	1 + 100
<p style="...">	1000	1000
#demo	100	100
.test	10	10
p .test1 .test2	21	1 + 10 + 10
#navbar p #demo	201	100 + 1 + 100

Category	Example	Points
Inline styles	<h1 style="color:pink">	1000
IDs	#navbar	100
Classes, pseudo-classes, attributes	.test :hover [href]	10
Elements, pseudo-elements	h1 ::before	1

Specificity in action

```
<style>
  p { color: red; }
  .test { color: green; }
  #demo { color: blue; }
</style>
<p>1. element only -> red</p>
```

```
<p class="test">2. + class -> green</p>
<p id="demo" class="test">3. + id -> blue</p>
<p id="demo" class="test" style="color: pink;">4. + inline -> pink</p>
```

browser output

1. element only -> red

2. + class -> green

3. + id -> blue

4. + inline -> pink

- **BIG IDEA** *Quiz: A = h1, B = h1 #content, C = <h1 id="content" style="color:pink">. Scores: A=1, B=101, C=1000. C wins.*

3. Inheritance

- *Children inherit certain styles from their parent -> less repeated code, consistent look.*
- *Inherited (usually):* color, font-family, font-size, text-align, line-height. *NOT inherited:* margin, padding, border, background.

```
<style>
  .parent { color: darkgreen; font-family: Arial, sans-serif; }
  h1 { text-transform: uppercase; }
  span { color: darkred; }     /* own rule beats inherited colour */
</style>
<div class="parent">
  <h1>Title</h1>
  <p>This is a paragraph. <span>And a span inside it.</span></p>
</div>
```

browser output

TITLE

This is a paragraph. And a span inside it.

Default browser styles

- Browsers have default CSS used when there is no other style information; silently inherited by all documents.
- Caution: defaults **differ between browsers** (an old Firefox page background was white, old IE about 5% gray) - that's why many sites use a CSS reset.

What the browser does (summary)

Step	Rule
1	find all rules that match the element
2	highest **specificity** wins
3	same specificity -> the **later** rule wins
4	no rule at all -> **inherit** from the parent, else use the default

- **TIP** !important after a value (e.g. color: red !important;) overrides normal specificity - use it very rarely, it makes CSS hard to debug.

13 Values, Units, Colours, Fonts & Text

In one line: *The small building blocks used inside every declaration.*

Values in CSS rules

Kind	Examples
colour names	black, blue, tomato, darkgreen
hex RGB	#A0A6AA (#RRGGBB), short #fff
rgb() / rgba()	rgb(255, 0, 0), rgba(0,0,0,0.5) (0.5 = half transparent)
pixels / points	12px, 10pt
inches / cm / mm	1in, 1cm, 1mm
relative (extra)	%, em, rem - scale with parent / root font size

```
<p style="color:#A0A6AA">hex #A0A6AA</p>
<p style="color:tomato">name tomato</p>
<p style="color:rgb(0,128,0)">rgb(0,128,0)</p>
<p style="font-size:12px">12px</p><p style="font-size:24px">24px</p>
```

browser output

hex #A0A6AA

name tomato

rgb(0,128,0)

12px

24px

Font rules

Property	Values
color	colour of the text
font-size	xx-small ... xx-large, smaller, larger, or a number like 16px

Property	Values
font-family	*comma-separated list; browser uses the FIRST available; end with a generic (serif, sans-serif, monospace)*
font-weight	normal, bold, bolder, lighter, *or* 100 ... 900
font-style	normal, italic, oblique
font-variant *(extra)*	small-caps

```
<style>
  .f { font-family: Verdana, Arial, sans-serif; font-size: 18px; }
</style>
<p class="f" style="font-weight:700">Bold 700</p>
<p class="f" style="font-style:italic">Italic</p>
<p class="f" style="font-size:xx-large">xx-large</p>
<p class="f" style="font-variant:small-caps">small caps text</p>
```

browser output

Bold 700

Italic

xx-large

SMALL CAPS TEXT

Text rules

Property	Values
text-decoration	none, underline, line-through, overline
text-align	left, right, center, justify
text-transform	uppercase, lowercase, capitalize
line-height *(extra)*	*space between lines, e.g. 1.5*

```
<p style="text-decoration:underline">underline</p>
<p style="text-decoration:line-through">line-through</p>
<p style="text-align:center">centred</p>
<p style="text-align:right">right aligned</p>
```

browser output

underline

~~line-through~~

centred

right aligned

The font shorthand

```
font: italic normal bold 12px Verdana;
/* is the same as: */
font-style: italic;
font-variant: normal;
font-weight: bold;
font-size: 12px;
font-family: Verdana;
```

- **TIP** *Shorthand order: style, variant, weight, size, family. Size and family are the required parts.*

14 Backgrounds & Borders

In one line: *Colours, images and lines around elements.*

Background properties

Property	Values
background-color	*solid colour*
background-image	url("back.gif")
background-repeat	repeat, repeat-x, repeat-y, no-repeat
background-attachment	scroll, fixed
background-position	top / center / bottom + left / center / right, *or* % / *px:* top left, -5px 50%
background-size *(extra)*	cover, contain

```
<div style="background-color:#FFF0C0; padding:14px">
  background-color: #FFF0C0</div>
<div style="background:linear-gradient(90deg,#9ecbee,#fff); padding:14px">
  (an image / gradient can be a background too)</div>
```

browser output

background-color: #FFF0C0

(an image / gradient can be a background too)

```
background: #FFF0C0 url("back.gif") no-repeat fixed top;
/* = color, image, repeat, attachment, position */
```

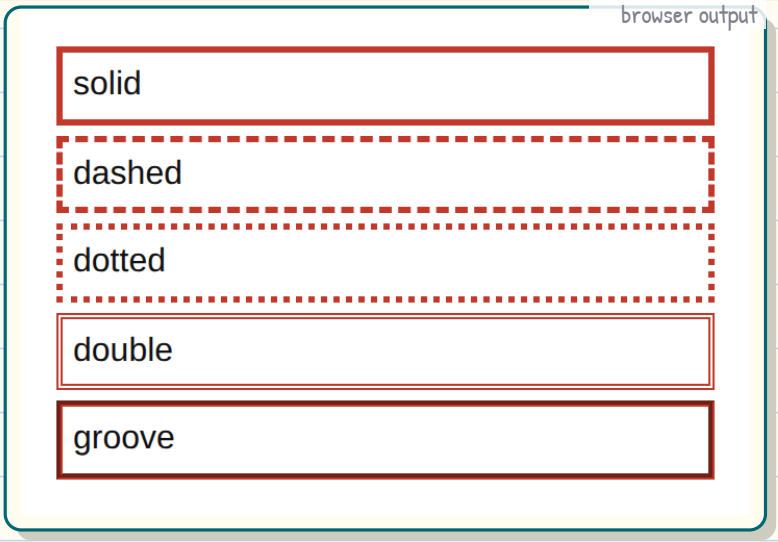
- **CAREFUL** *With the shorthand, set the colour AND image carefully - some old browsers dropped one of them. The shorthand also resets anything you did not write.*

Border properties

Property	Values
border-width	thin, medium, thick, or 10px
border-color	name or RGB
border-style	none, hidden, dotted, dashed, solid, double, groove, ridge, inset, outset

```
<style>div{margin:5px;padding:5px;border-width:3px;border-color:#c0392b}</style>
<div style="border-style:solid">solid</div>
<div style="border-style:dashed">dashed</div>
<div style="border-style:dotted">dotted</div>
<div style="border-style:double">double</div>
<div style="border-style:groove">groove</div>
```

browser output



Border shorthand and single sides

```
<p style="border: 1px solid red; padding:6px">border: 1px solid red;</p>
<p style="border-bottom: 4px solid blue; padding:6px">only border-bottom</p>
<p style="border-left: 8px solid green; padding:6px">only border-left</p>
```

browser output

border: 1px solid red;

only border-bottom

only border-left

- border: width style color; is the same as writing the three properties separately.
- Each side separately: border-top, border-right, border-bottom, border-left (or border-top-style, border-left-color ...).

15 The Box Model, Sizes & Overflow

In one line: Every element is a box: content + padding + border + margin.

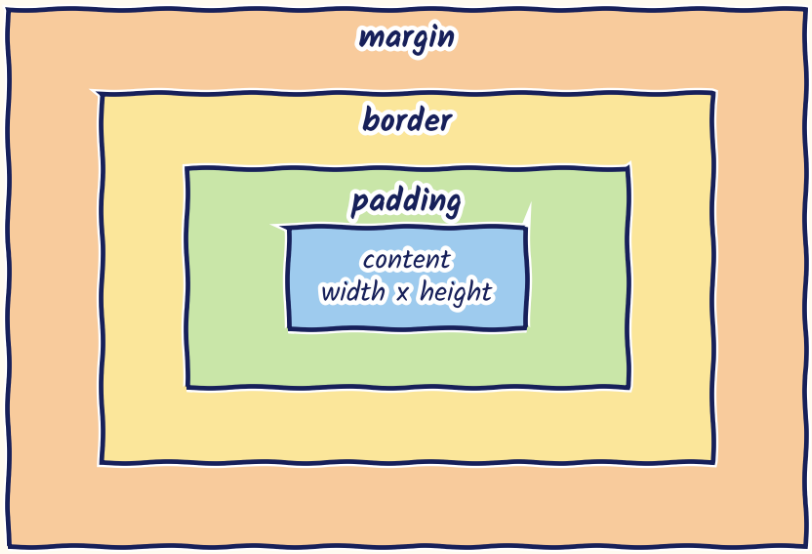


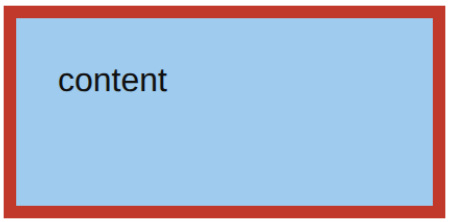
Figure: the CSS box model, from the inside out.

The four layers

Layer	Meaning
content	the text / image itself (width x height)
padding	space BETWEEN the content and the border (inside)
border	line around the padding
margin	space OUTSIDE the border, pushes other elements away

```
<style>
 .box{background:#9ecbee;width:160px;height:50px;padding:20px;
    border:6px solid #c0392b;margin:20px;}
</style>
<div class="box">content</div>
```

browser output



Width and height

- width: 200px; height: 100px; set the size of the CONTENT box.
- Height is usually decided by the content itself. Common mistake: setting height on tables or table cells.
- Not all elements / browsers follow width / height - inline elements ignore them.
- Limits: min-width, max-width, min-height, max-height.

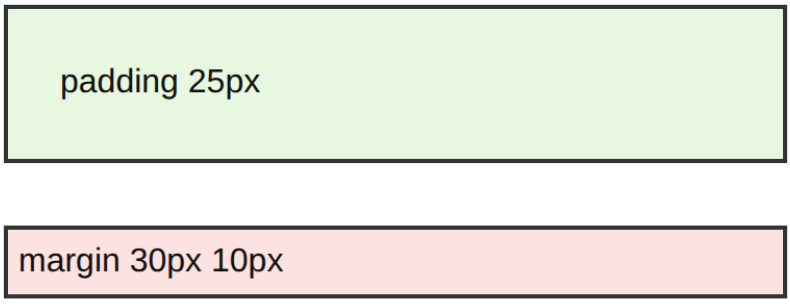
Margin and padding shorthand

Code	Meaning
margin: 5px;	all four sides 5px
margin: 10px 20px;	top + bottom 10px, left + right 20px
margin: 1px 3px 5px 7px;	top, right, bottom, left (clockwise)
margin-top padding-left ...	single sides

- Same shorthand for **padding**. margin: 0 auto; centres a block with a set width.

```
<div style="border:2px solid #333; padding:25px; margin:10px;
  background:#e8f7e0">padding 25px</div>
<div style="border:2px solid #333; padding:5px; margin:30px 10px;
  background:#fde2e2">margin 30px 10px</div>
```

browser output

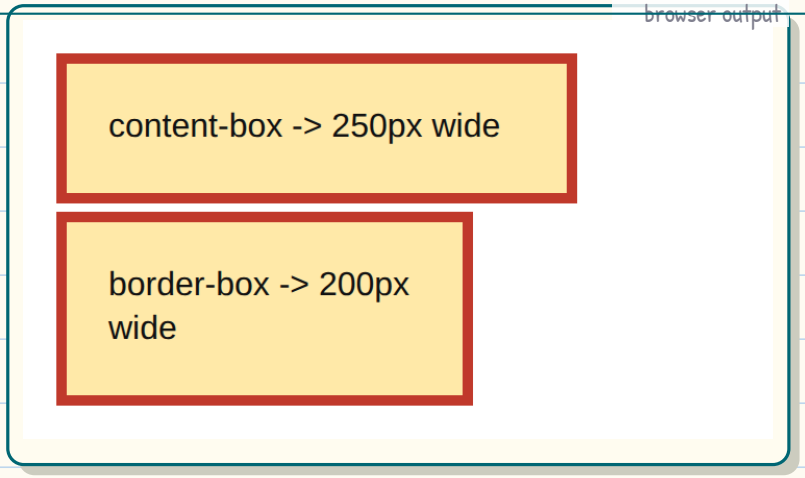


box-sizing: content-box vs border-box

	content-box (default)	border-box
width means	width of the CONTENT only	content + padding + border
real size of width:200px + padding:20px + border:5px	200 + 40 + 10 = **250px**	stays **200px**
when to use	default	easier layouts - recommended with * { box-sizing: border-box; }

```
<style>.b{width:200px;padding:20px;border:5px solid
  #c0392b;background:#ffe9a8;margin:4px}</style>
<div class="b" style="box-sizing:content-box">content-box -> 250px wide</div>
<div class="b" style="box-sizing:border-box">border-box -> 200px wide</div>
```

browser output



Overflow

Value	Result when content is too big
visible	*default - spills outside the box*
hidden	*extra content is clipped, no scrollbars*
scroll	*clipped, scrollbars ALWAYS shown*
auto	*scrollbars only when needed*
overflow-x overflow-y	*control each direction separately*

```
<style>.o{width:150px;height:45px;border:1px solid
  #04AA6D;margin:4px;display:inline-block;vertical-align:top}</style>
<div class="o" style="overflow:visible">visible: Already have an account? Sign in here
  please.</div>
<div class="o" style="overflow:hidden">hidden: Already have an account? Sign in here
  please.</div>
<div class="o" style="overflow:scroll">scroll: Already have an account? Sign in
  here.</div>
<div class="o" style="overflow:auto">auto: Already have an account? Sign in here
  please.</div>
```

browser output

visible: Already have an account? Sign in here please.

hidden: Already have an account? Sign in

scroll: Already have an account? Sign in

auto: Already have an account? Sign in

16 Display, Visibility, Float & Positioning

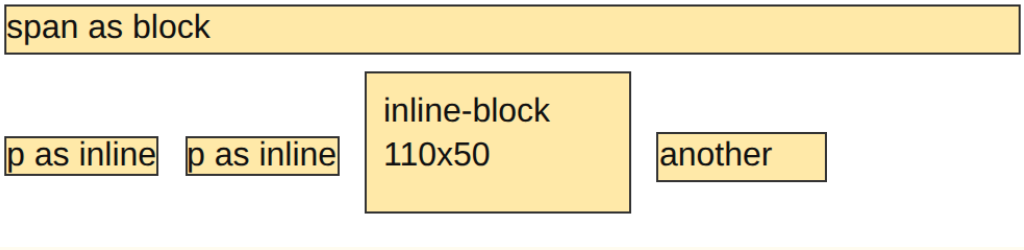
In one line: *How elements are shown, taken out of the normal flow, and layered.*

display

Value	Behaviour
block	*new line before/after, full width*
inline	*stays in the line, only needed width, ignores width / height*
inline-block	*stays in the line BUT accepts width, height, margin, padding*
none	*removed completely - takes NO space*

```
<style>.bx{background:#ffe9a8;border:1px solid #333;margin:4px}</style>
<span class="bx" style="display:block">span as block</span>
<p class="bx" style="display:inline">p as inline</p>
<p class="bx" style="display:inline">p as inline</p>
<div class="bx"
   style="display:inline-block;width:110px;height:50px;padding:8px">inline-block
      110x50</div>
<div class="bx" style="display:none">I am hidden (display:none)</div>
<div class="bx" style="display:inline-block;width:80px">another</div>
```

browser output

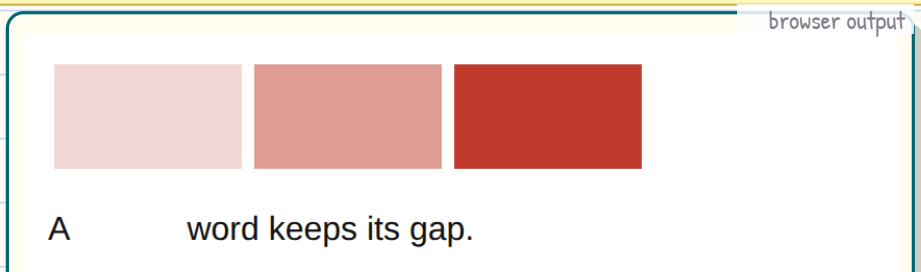


visibility and opacity

Rule	Effect
visibility: hidden	not drawn BUT still takes its space (acts like opacity 0)
display: none	not drawn and takes NO space
opacity: 0..1	transparency: 1 = solid (default), 0.5 = 50% see-through, 0 = invisible

```
<style>
.s{display:inline-block;width:90px;height:50px;
   background:#c0392b;margin:3px}
</style>
<div class="s" style="opacity:0.2"></div><div class="s" style="opacity:0.5"></div><div
   class="s"></div>
<p>A <span style="visibility:hidden">hidden</span> word keeps its gap.</p>
```

browser output

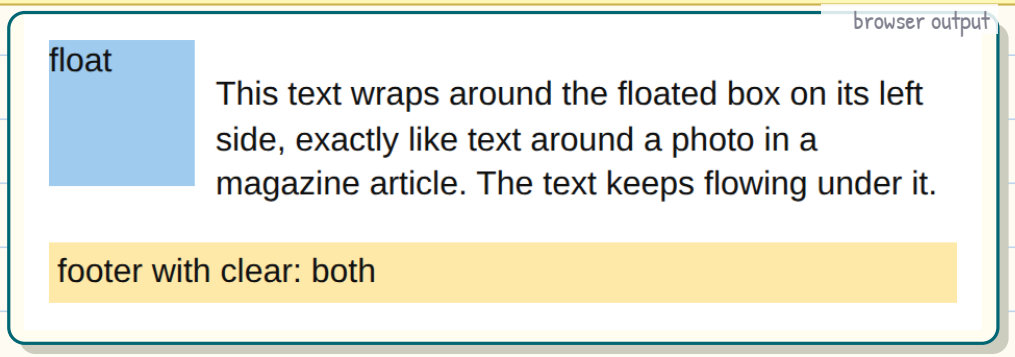


float and clear

- float: left | right lifts an element out of the flow and pushes it to a side; text **wraps around** it.
- clear: left | right | both = no floating elements allowed on that side (used on a footer after floats).
- Float was the old layout method; for page layouts use Flexbox / Grid.

```
<div style="width:70px;height:70px;background:#9ecbee;
    float:left;margin-right:10px">float</div>
<p>This text wraps around the floated box on its left side, exactly like text
around a photo in a magazine article. The text keeps flowing under it.</p>
<div style="clear:both;background:#ffe9a8;padding:4px">footer with clear: both</div>
```

browser output

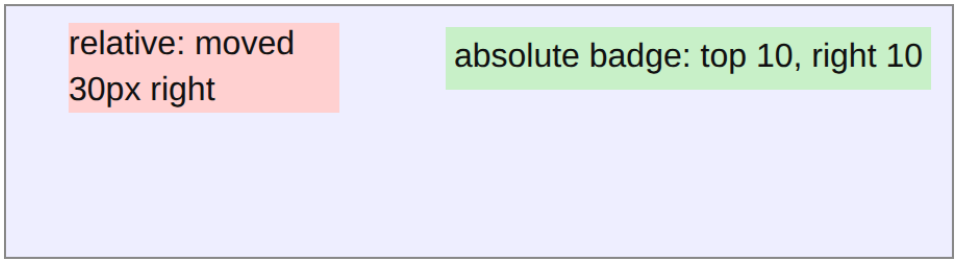


position

Value	Behaviour
static	DEFAULT - normal flow; top / left / z-index have NO effect
relative	stays in flow (keeps its space) but can be moved with top / left...
absolute	REMOVED from the flow; placed relative to the nearest POSITIONED ancestor (else the page)
fixed	fixed to the browser window - ignores scrolling (nav bars, chat buttons)
sticky	normal flow until a scroll point, then sticks

```
<style>
 .wrap{position:relative;height:120px;background:#eef;border:1px solid #888}
 .r{position:relative;left:30px;top:8px;background:#ffd0d0;width:130px}
 .a{position:absolute;top:10px;right:10px;background:#c8f0c8;padding:4px}
</style>
<div class="wrap">
  <div class="r">relative: moved 30px right</div>
  <span class="a">absolute badge: top 10, right 10</span>
</div>
```

browser output



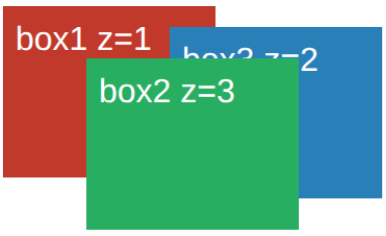

```
.card  { position: relative; }     /* the reference point */
.badge { position: absolute; top: 10px; right: 10px; }
.button { position: fixed; right: 20px; bottom: 20px; }
nav    { position: sticky; top: 0; }
```

z-index - stacking order

- z-index decides which element is IN FRONT (higher number = on top).
- It only works on **positioned** elements (relative, absolute, fixed, sticky) - ignored on static.

```
<style>.z{position:absolute;width:90px;height:70px;color:#fff;padding:6px}</style>
<div style="position:relative;height:120px">
 <div class="z" style="background:#c0392b;left:0;top:0;z-index:1">box1  z=1</div>
 <div class="z" style="background:#27ae60;left:40px;top:25px;z-index:3">box2
    z=3</div>
 <div class="z" style="background:#2980b9;left:80px;top:10px;z-index:2">box3
    z=2</div>
</div>
```

browser output



top / left / bottom / right and vertical-align

- top, left, bottom, right = *offsets for positioned elements.*
- vertical-align = *vertical alignment of inline elements and table cells:*
  baseline, sub, super, top, middle, bottom, text-top, text-bottom, *or a number.*

Part D

CSS Layout

17 Flexbox

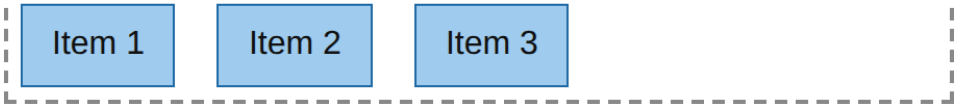
In one line: Flexbox arranges items in ONE direction (a row OR a column) with easy alignment and spacing.

What is layout? and the main idea

- **Layout** = where elements appear and how much space they use. HTML = the content, CSS layout = the arrangement.
- Modern layout mainly uses **Flexbox** and **Grid** (plus display and position). Start with the question: *Where should this element go?*
- Set the PARENT to display: flex -> it becomes the **flex container**; its direct children become **flex items**.

```
<div class="c" style="display:flex; gap:20px;">
  <div>Item 1</div><div>Item 2</div><div>Item 3</div>
</div>
```

browser output



- **FACT** Picture colours were added only to make the boxes visible. Only direct children become flex items.

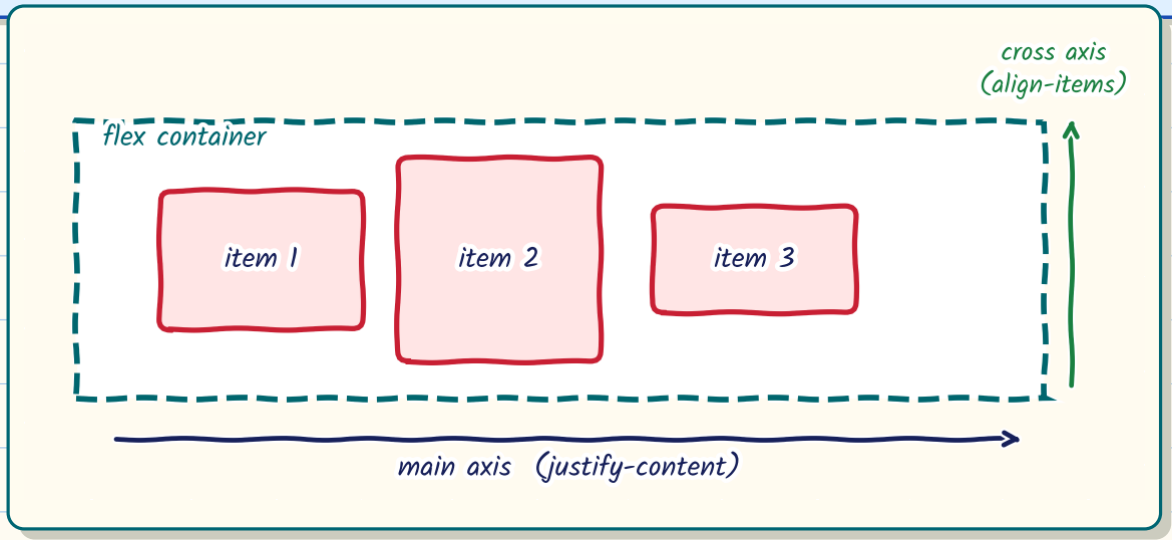


Figure: main axis (direction of the row) and cross axis (across it).

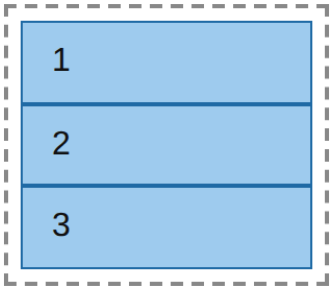
flex-direction

Value	Result
row (default)	left to right
row-reverse	right to left
column	top to bottom
column-reverse	bottom to top

```
.container { display: flex; flex-direction: row; }
.column    { display: flex; flex-direction: column; }
```

```
<div class="c" style="display:flex;flex-direction:column;width:140px">
  <div>1</div><div>2</div><div>3</div></div>
```

browser output



justify-content - along the MAIN axis

Value	Effect
flex-start	items at the beginning
flex-end	items at the end
center	items centred
space-between	equal space BETWEEN items (first and last touch edges)
space-around / space-evenly	space distributed around / evenly

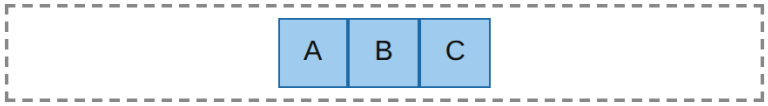
```
.container { display: flex; justify-content: flex-start; }
```

justify-content: flex-start



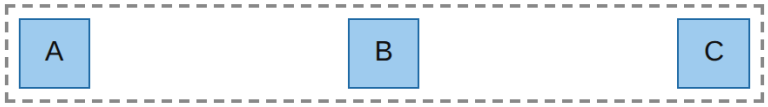

```
.container { display: flex; justify-content: center; }
```

justify-content: center



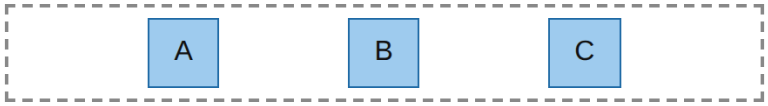

```
.container { display: flex; justify-content: space-between; }
```

justify-content: space-between




```
.container { display: flex; justify-content: space-evenly; }
```

justify-content: space-evenly

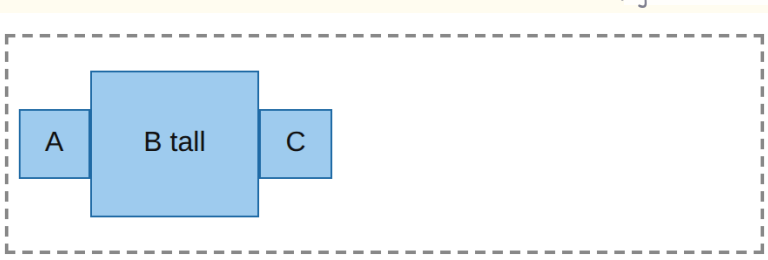


align-items – along the CROSS axis

Value	Effect
stretch (default)	items fill the cross-axis space
flex-start / flex-end	toward the two edges
center	centred (vertical centring in a row)

```
.container { display: flex; height: 120px; align-items: center; }
```

align-items: center

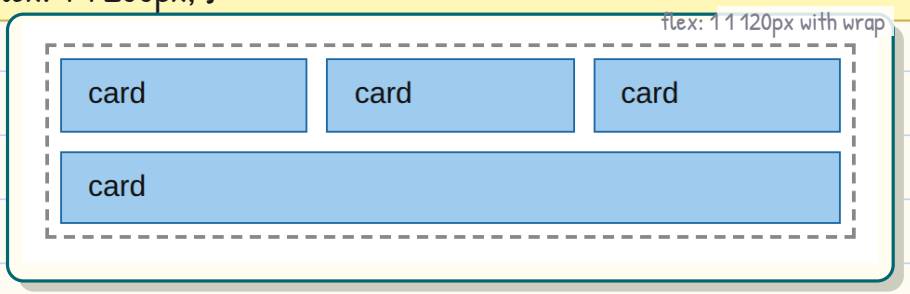


- **TIP** Perfect centring: display:flex; justify-content:center; align-items:center;

gap, flex-wrap and the flex shorthand

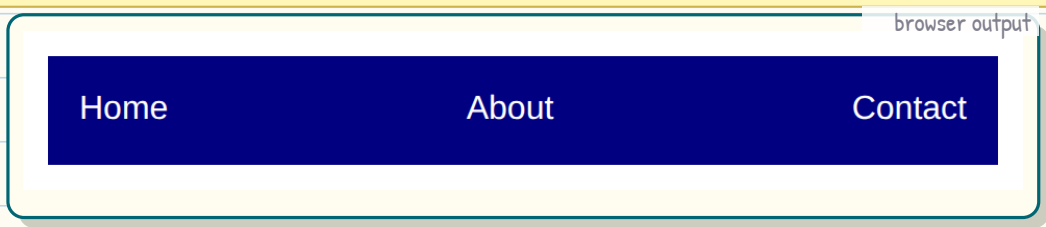
- gap = space between items (no margin tricks needed).
- flex-wrap: wrap = items move to a NEW line when there is no room.
- flex: grow shrink basis - flex: 1 1 200px = can grow, can shrink, ideal width 200px.

```
.container { display: flex; gap: 20px; flex-wrap: wrap; }
.card      { flex: 1 1 200px; }
```



Complete example: navigation bar

```
<nav class="nav">
  <a href="#">Home</a>
  <a href="#">About</a>
  <a href="#">Contact</a>
</nav>
<style>
.nav { display: flex; justify-content: space-between;
       align-items: center; gap: 20px; padding: 15px;
       background: navy; }
.nav a { color: white; text-decoration: none; }
</style>
```



- **TIP** Experiment: remove justify-content and watch the links bunch to the left.

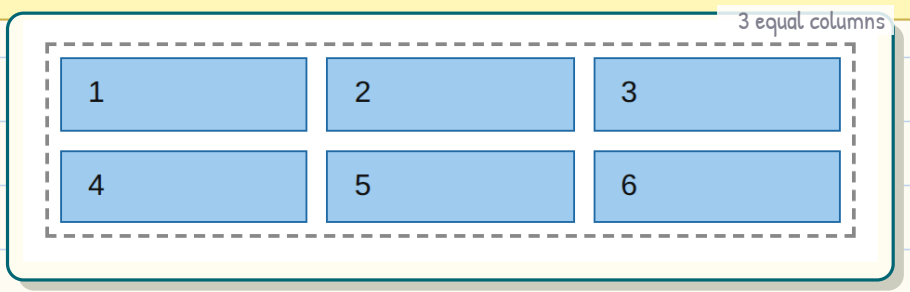
18 CSS Grid

In one line: Grid lays items out in TWO dimensions: rows AND columns.

The main idea

- Set the parent to display: grid. Define grid-template-columns and grid-template-rows; gap adds space.
- fr = one fraction of the available space. repeat(3, 1fr) = three equal columns.

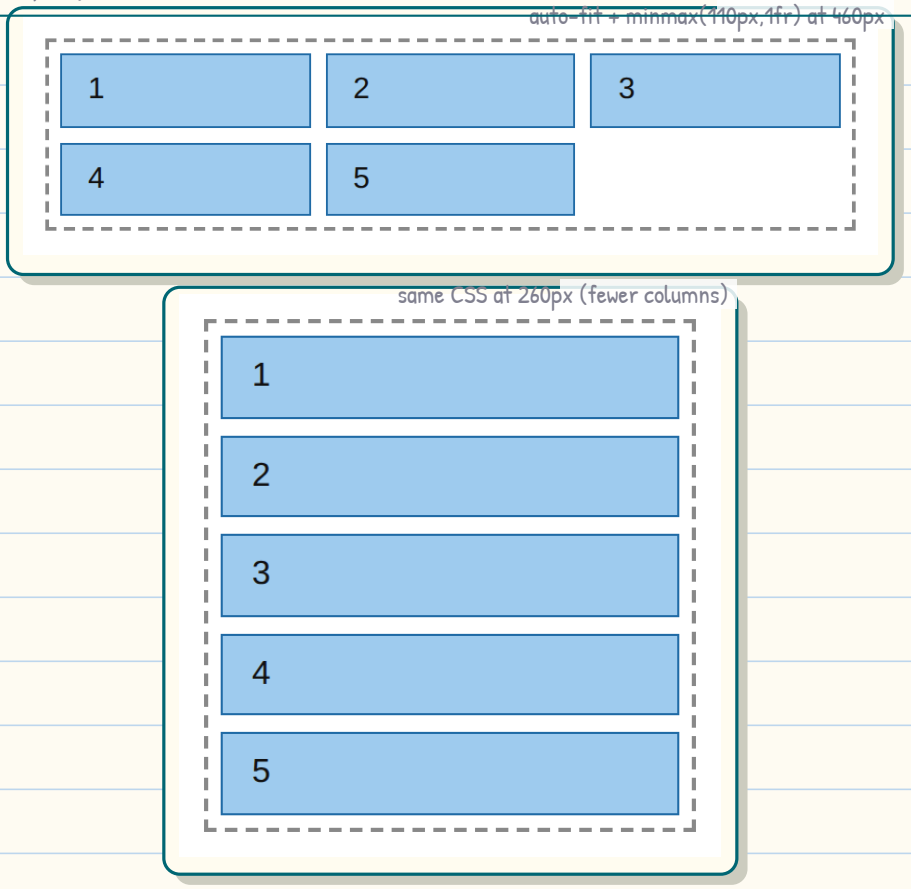
```
.container {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 20px;
}
```



repeat() and minmax()

```
.grid { display: grid;
        grid-template-columns: repeat(3, 1fr); gap: 20px; }
.responsive-grid { display: grid;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr)); }
```

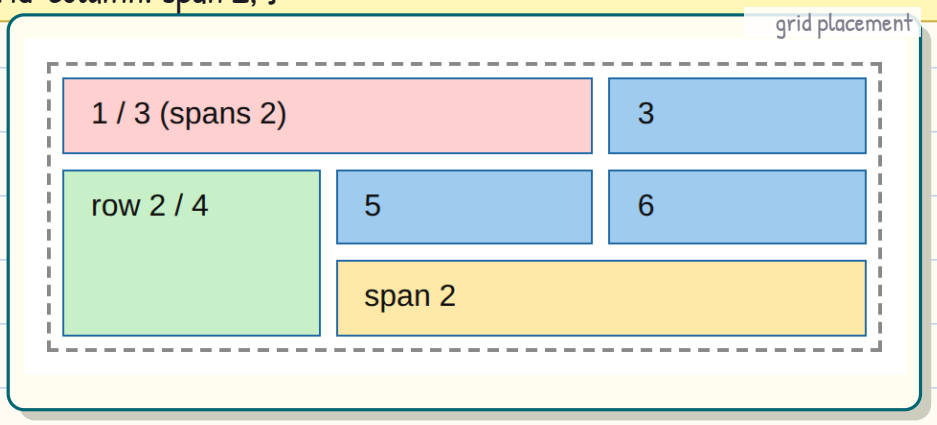
- minmax(200px, 1fr) = at least 200px, at most one fraction. auto-fit = fit as many columns as possible -> **responsive without media queries**.



Placing items: lines and span

Property	Meaning
grid-column: 1 / 3;	start at column line 1 and end at column line 3 (covers 2 columns)
grid-row: 2 / 4;	start at row line 2, end at row line 4
grid-column: span 2;	cover 2 columns from where it is

```
.item1 { grid-column: 1 / 3; }
.item2 { grid-row: 2 / 4; }
.item3 { grid-column: span 2; }
```



Flexbox or Grid?

	Flexbox	Grid
dimensions	ONE (row OR column)	TWO (rows AND columns)
best for	navbars, button groups, card rows, centring	page layouts, galleries, card grids
parent rule	display: flex	display: grid
sizing driven by	the content	the grid you define

- **REMEMBER** They work together: Grid for the overall page, Flexbox inside each part.

19 Responsive Design & Media Queries

In one line: One page that adapts to desktop, tablet and phone.

What is responsive design?

- A **responsive** page adapts its layout to different screen sizes: desktop may show cards in a row, mobile stacks them.
- Tools: flexible units (%, fr), Flexbox, Grid, and **media queries**.
- Always add the viewport meta tag in <head> so phones don't zoom out:

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

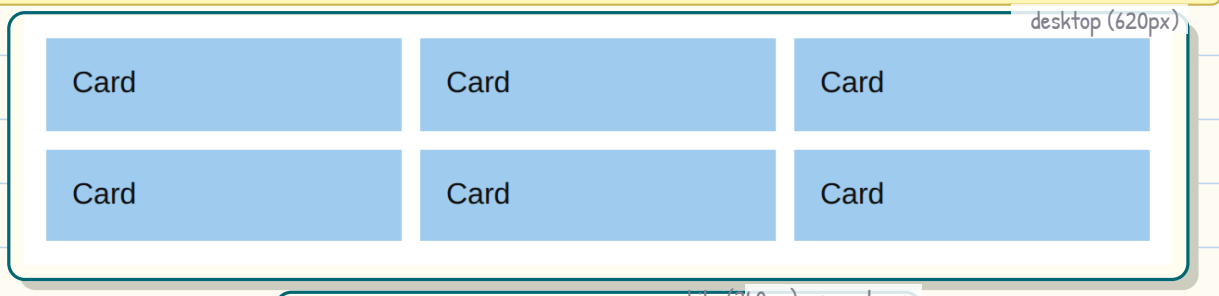
Media query syntax

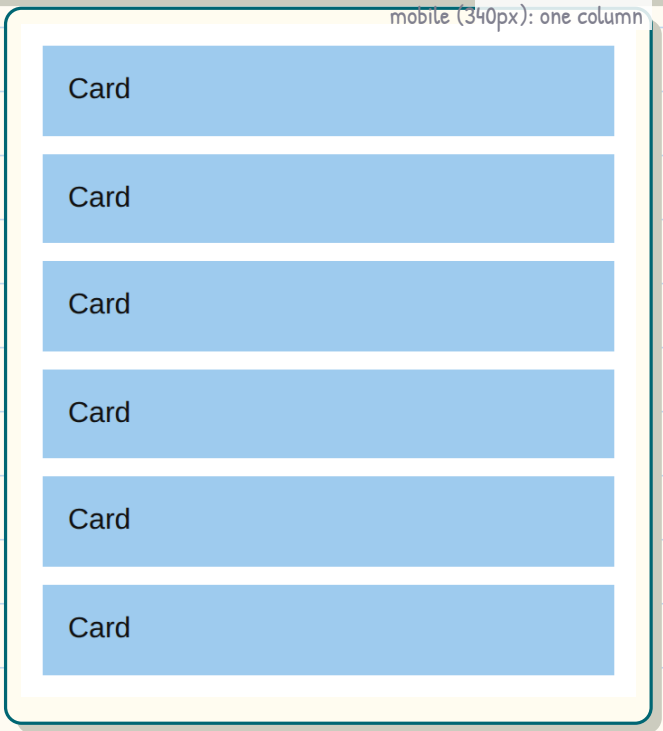
```
@media media-type and (condition) {
    /* CSS rules that apply only when the condition is true */
}
```

Condition	Meaning
max-width: 600px	screen is 600px wide OR LESS (phones)
min-width: 768px	768px wide OR MORE
max-height min-height	same for height
orientation: landscape | portrait	device orientation

Example: cards become one column

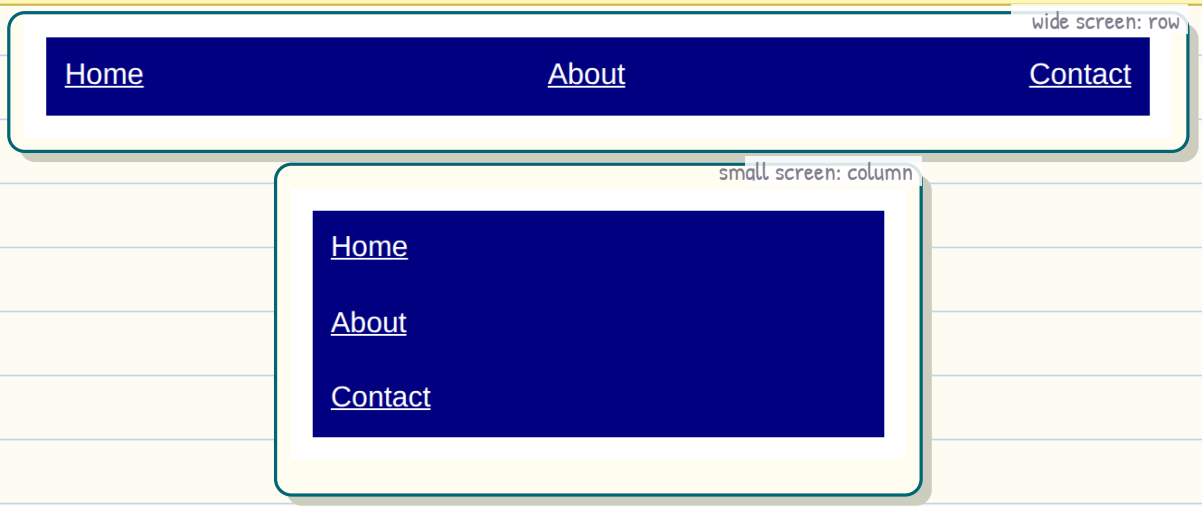
```
<style>
.cards { display: grid; grid-template-columns: repeat(3, 1fr);
        gap: 10px; }
.cards div { background: #9ecbee; padding: 14px; }
@media (max-width: 600px) {
  .cards { grid-template-columns: 1fr; }
}
</style>
<div class="cards"><div>Card</div><div>Card</div><div>Card</div>
<div>Card</div><div>Card</div><div>Card</div></div>
```





Example: responsive navigation

```
<style>
.nav { display: flex; justify-content: space-between; gap: 20px;
      background: navy; padding: 10px; }
.nav a { color: white; }
@media (max-width: 600px) {
  .nav { flex-direction: column; }
}
</style>
<nav class="nav"><a href="#">Home</a><a href="#">About</a><a
   href="#">Contact</a></nav>
```



- The browser applies the media-query rule ONLY while the condition is true.
- **Mobile-first** style: write the phone layout first, then add `@media (min-width: ...)` for bigger screens.

Feature queries: @supports

```
.container { display: block; }    /* fallback */
@supports (display: grid) {
  .container { display: grid; }    /* used if Grid is supported */
}
```

- `@supports` checks whether the browser understands a CSS feature -> modern browsers use Grid, older ones use the fallback.

Layout mistakes beginners make

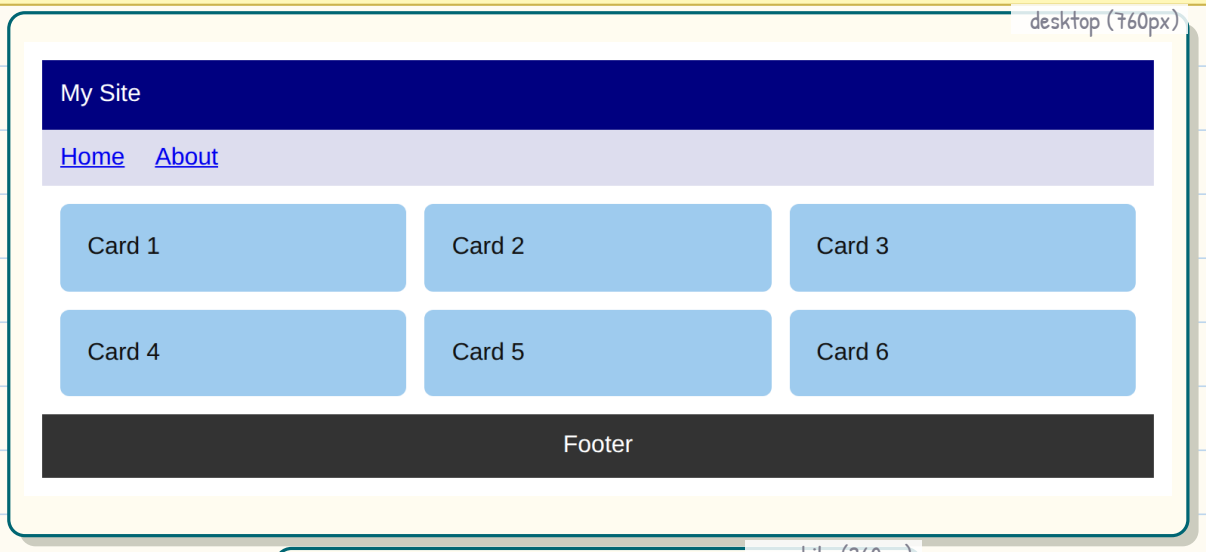
Mistake	Fix
absolute positioning for the whole page	use Flexbox / Grid for layout
forgetting position:absolute leaves the normal flow	remember siblings will move up
fixed widths that don't fit small screens	use %, fr, max-width
not using gap, flex-wrap, Grid	use them instead of margin hacks
fixing everything with margin	check the PARENT: display, width, direction, gap
media query outside the CSS / style block	keep it inside <style> or the .css file

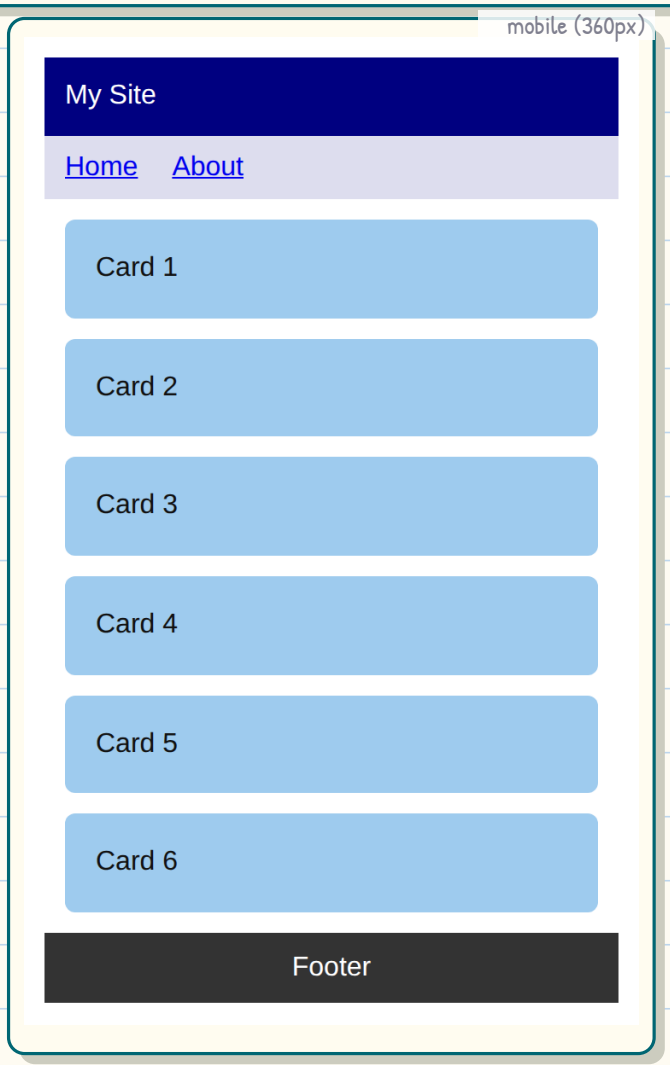
- **TIP** When a layout breaks, inspect the PARENT first: display, width, direction, gap and positioning.

Practice: build a page layout (10-15 minutes)

```
<style>
* { box-sizing: border-box; margin: 0; }
body { font-family: Arial; }
header { background: navy; color: white; padding: 12px; }
.nav { display: flex; gap: 20px; padding: 8px 12px; background: #dde; }
.cards { display: grid; grid-template-columns: repeat(3, 1fr);
        gap: 12px; padding: 12px; }
.cards div { background: #9ecbee; padding: 18px; border-radius: 6px; }
footer { background: #333; color: white; padding: 10px;
        text-align: center; }
@media (max-width: 700px) {
  .cards { grid-template-columns: 1fr; }
}
</style>
<header>My Site</header>
<nav class="nav"><a href="#">Home</a><a href="#">About</a></nav>
<main class="cards"><div>Card 1</div><div>Card 2</div><div>Card 3</div>
```

```
<div>Card 4</div><div>Card 5</div><div>Card 6</div></main>
<footer>Footer</footer>
```





Part E

CSS3 Features

20 CSS3 Visual Effects

In one line: *Rounded corners, gradients, shadows and transparency make modern, attractive pages.*

Rounded corners – border-radius

```
.card { width: 300px; padding: 20px; border: 2px solid black;
        border-radius: 15px; }
```

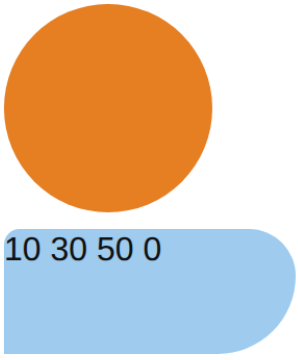
border-radius: 15px

Rounded card (radius 15px)

Code	Meaning
border-radius: 20px;	all four corners
border-radius: 10px 20px 30px 40px;	top-left, top-right, bottom-right, bottom-left
border-top-left-radius ...	single corner properties
border-radius: 50%;	circle (when width = height)

```
<div style="width:100px;height:100px;border-radius:50%;background:#e67e22"></div>
<div style="width:140px;height:60px;margin-top:8px;background:#9ecbee;
   border-radius:10px 30px 50px 0">10 30 50 0</div>
```

browser output

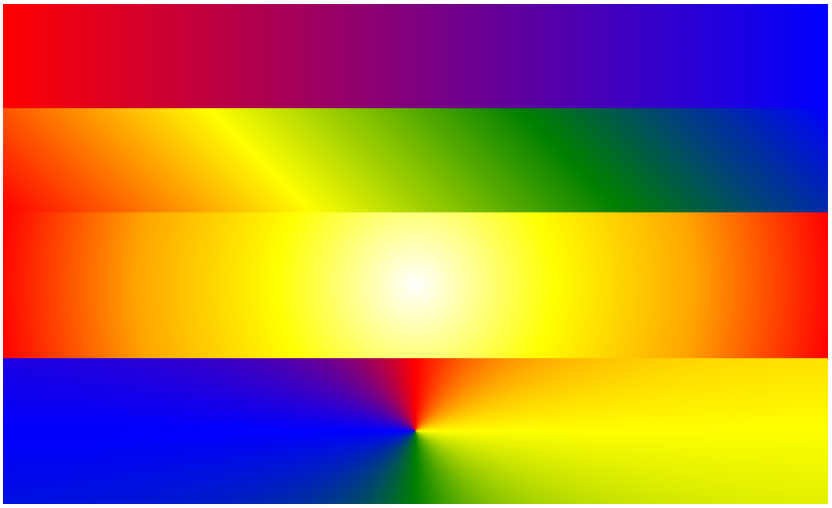


Gradients

Type	Example	Idea
linear	linear-gradient(red, blue)	colour changes along a LINE; to right, to bottom, or an angle 45deg; many colours allowed
radial	radial-gradient(circle, white, yellow, orange, red)	spreads OUTWARD from a centre
conic	conic-gradient(red, yellow, green)	colours rotate around a centre

```
<div style="height:50px;background:linear-gradient(to right, red, blue)"></div>
<div style="height:50px;background:linear-gradient(45deg, red, yellow, green,
  blue)"></div>
<div style="height:70px;background:radial-gradient(circle, white, yellow, orange,
  red)"></div>
<div style="height:70px;background:conic-gradient(red, yellow, green, blue, red)"></div>
```

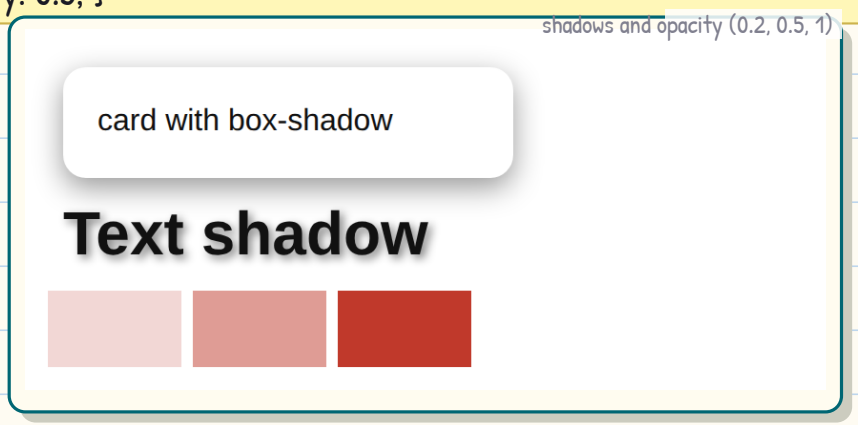
browser output



Shadows and opacity

Property	Syntax	Notes
box-shadow	x-offset y-offset blur spread color	shadow around an ELEMENT (cards, buttons, nav, modals)
text-shadow	x y blur color	shadow around TEXT
opacity	0 to 1	0 = invisible, 1 = solid; affects the whole element

```
.card { border-radius: 12px; box-shadow: 0 5px 15px #999; }
h1   { text-shadow: 2px 2px 5px gray; }
img  { opacity: 0.5; }
```



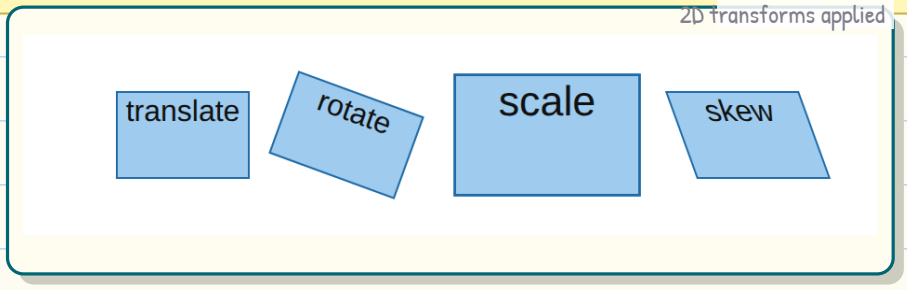
21 Transform, Transition & Animation

In one line: *Transform = WHAT changes. Transition = HOW SMOOTHLY. Animation = changes through MANY stages.*

transform (2D)

Function	Does	Example
translate(x, y)	moves (translateX, translateY)	translate(50px, 20px)
rotate(angle)	rotates	rotate(20deg)
scale(n)	resizes: 1 = original, 1.2 = 120%, 0.5 = 50%	scale(1.1)
skew(angle)	slants (skewX, skewY)	skew(20deg)

```
.box { transform: rotate(20deg); }
.box:hover { transform: translateY(-5px); }
img:hover { transform: scale(1.1); }
```

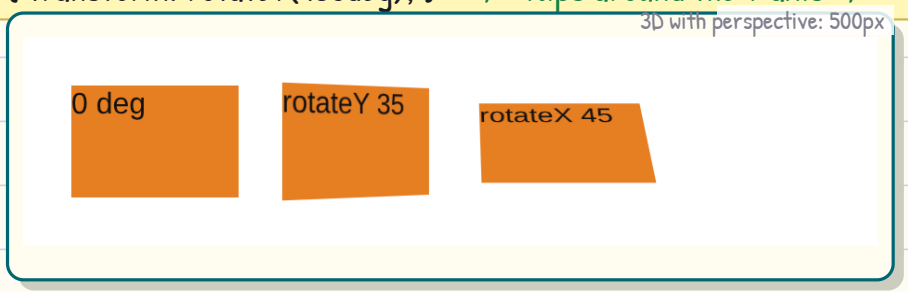


- **FACT** Transform changes the VISUAL state without changing the normal document flow (other elements do not move).

2D vs 3D and perspective

- 2D uses X and Y: translate, rotate, scale, skew. **3D adds the Z axis:** translateZ(), rotateX(), rotateY(), rotateZ().
- perspective: 500px on the parent sets how deep the 3D effect looks (distance between your eye and the Z=0 plane; smaller = more dramatic).

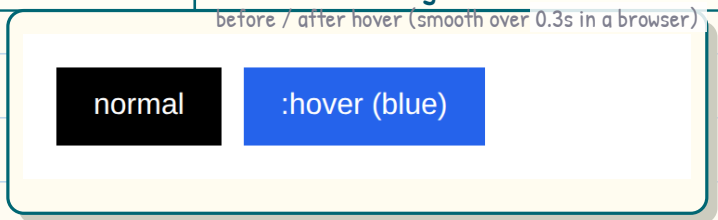
```
.container { perspective: 500px; }
.card:hover { transform: rotateY(180deg); }     /* flips around the Y axis */
```



transition - smooth change between two states

```
button { background: black; color: white; transition: 0.3s; }
button:hover { background: blue; }
```

Property	Meaning
transition-property	which property to animate (all, background...)
transition-duration	how long (0.3s)
transition-timing-function	speed pattern: ease, linear, ease-in, ease-out, ease-in-out
transition-delay	wait before starting
shorthand	transition: background 0.3s ease;



- Without a transition the change is instant; WITH it the change is gradual.

- Popular combo - lift on hover:

```
.button { transition: all 0.2s ease; }
.button:hover { transform: translateY(-2px); }
```

@keyframes - animation through stages

```
@keyframes mymove {
  from { top: 0px; }
  to   { top: 200px; }
}
div {
  width: 100px; height: 100px; background: red;
  position: relative;
  animation: mymove 2s infinite;
}
```

Property	Meaning
animation-name	name of the @keyframes
animation-duration	length of one cycle
animation-delay	wait before start
animation-iteration-count	how many times (infinite)
animation-direction	normal, reverse, alternate
animation-timing-function	speed pattern

```
@keyframes slide {
  0%   { left: 0;    }      /* start */
  50%  { left: 120px; }     /* move  */
  100% { left: 0;    }      /* return */
}
```

- Use **percentages** for multi-step animation: 0% start, 50% middle, 100% end.
- Animation runs **automatically**; a transition needs a **state change** such as :hover.

Transition vs animation

	Transition	Animation
trigger	state change (:hover, :focus)	can run automatically
stages	from state A to state B	many stages with @keyframes
control	duration, timing, delay	also repeat count, direction

22 Web Fonts, Icon Fonts & CSS Variables

In one line: Use any font, scalable icons and reusable values.

Web fonts - @font-face

```
@font-face {
  font-family: MyFont;
  src: url('myfont.woff2');
}
body { font-family: MyFont; }
```

- Web fonts let a site use fonts that are NOT installed on the visitor's computer.

Format	Name
.woff2	Web Open Font Format 2 - best compression, preferred for modern sites
.woff	Web Open Font Format
.ttf	TrueType Font
.otf	OpenType Font

Icon fonts

```
<head>
  <script src="https://kit.fontawesome.com/a076d05399.js"></script>
</head>
<body>
  <i class="fas fa-heart"></i>
  <i class="fas fa-car"></i>
```

```
</body>
```

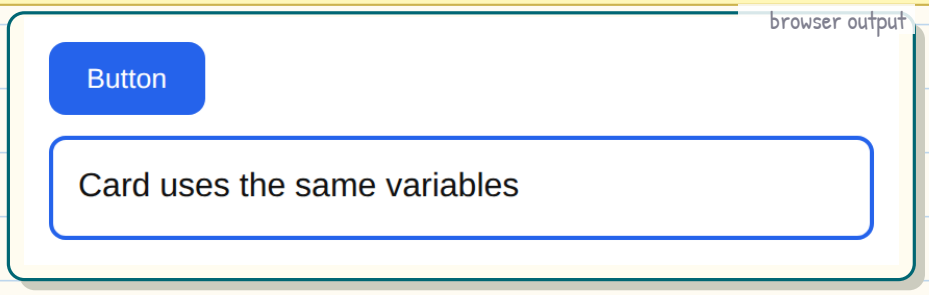
- Icons are drawn as font characters (Font Awesome, Material Icons) -> resize and recolour them with CSS (font-size, color).

CSS custom properties (variables)

Step	How
define (name starts with --)	:root { --primary-color: #2563eb; }
use with var()	button { background: var(--primary-color); }
:root	the top of the document -> variable available everywhere

```
<style>
:root { --primary-color: #2563eb; --radius: 8px; }
button { background: var(--primary-color); color: white;
        border: 0; padding: 10px 18px; border-radius: var(--radius); }
.card { border: 2px solid var(--primary-color);
       border-radius: var(--radius); padding: 12px; margin-top: 10px; }
</style>
<button>Button</button>
<div class="card">Card uses the same variables</div>
```

browser output



- **Why?** Without variables the same value is repeated all over. With variables: **change once -> updates everywhere** (consistency + maintainability).

23 Compatibility: Prefixes, Fallbacks & Testin

In one line: Make sure the page works in every target browser.

Vendor prefixes

Prefix	Browser engine
-webkit-	Chrome, Safari
-moz-	Firefox
-ms-	old Internet Explorer / Edge
-o-	old Opera

```
-webkit-transform: rotate(20deg);    /* old way */
transform: rotate(20deg);            /* modern, standard way */
```

- Prefixes added new / experimental features early. **Today they are usually NOT needed** for most CSS. Don't add them to everything; they may still appear in legacy code.

Cross-browser compatibility

- Browsers (Chrome, Firefox, Edge, Safari) differ in CSS support, rendering and **default styles**.
- **How to improve:** use standard CSS, check browser compatibility (caniuse.com), provide **fallbacks**, use **feature queries** (@supports), test in several browsers, use responsive design, avoid unnecessary experimental features.

- **BIG IDEA** Standard CSS + Fallback + Test = a page that works everywhere.

Part F

Revision

24 Final Revision & Practice

In one line: *One place for quick review before the exam.*

HTML cheat sheet

Group	Tags
structure	<!DOCTYPE html> <html> <head> <body> <title> <meta>
text	<h1>-<h6> <p> <mark> <small> <del> <ins> <sub> <sup> <pre> <hr>
links / media	<a href> <img src alt> <audio> <video> <source> <iframe>
lists	<dl> <dt> <dd>
tables	<table> <tr> <th> <td> <thead> <tbody> <tfoot> <colgroup> <col> + colspan rowspan
semantic	<header> <nav> <main> <section> <article> <aside> <footer>
generic	(block) (inline)
forms	<form action method> <input type> <textarea> <select><option> <button> <label for>

CSS cheat sheet

Topic	Remember
rule	selector { property: value; }
selectors	p type, * all, .class, #id, div p descendant, div > p child, h2 + p next sibling, h2 ~ p later siblings, :hover, ::before
specificity	inline 1000 > id 100 > class 10 > element 1; tie -> later wins
box model	content + padding + border + margin; box-sizing: border-box
display	block inline inline-block none
position	static relative absolute fixed sticky + z-index
flex	display:flex; flex-direction; justify-content (main); align-items (cross); gap; flex-wrap
grid	display:grid; grid-template-columns: repeat(3,1fr); gap; grid-column: 1/3
responsive	@media (max-width: 600px) { ... }, @supports

Topic	Remember
CSS3	border-radius gradients box-shadow transform transition @keyframes var(--x)

Quick review questions (from the lectures) - with answers

Question	Answer
What does display: flex do?	makes the element a flex container; its children become flex items laid out in a row (or column)
relative vs absolute?	relative keeps its space and moves from its normal spot; absolute leaves the flow and positions against the nearest positioned ancestor
When use fixed?	for things that stay on screen while scrolling: nav bars, chat buttons
What does overflow: auto do?	scrollbars appear only when content is too big
Flexbox vs Grid?	Flexbox = one dimension (row OR column); Grid = two (rows AND columns)
What does gap do?	adds space between flex / grid items
Why media queries?	to apply different CSS depending on screen size - responsive design
Card layout on a small screen?	3 columns become 1 column (stacked)
Transform / transition / animation?	what changes / how smoothly / how through many stages
Class vs ID?	class . many elements; id # one unique element
GET vs POST?	GET data in URL (non-private); POST data in body (sensitive / large)
1-tier / 2-tier / 3-tier?	all in one machine / client + data server / UI, logic, data on separate machines

Common mistakes and fixes

Mistake	Fix
wrongly nested tags ..	close the inner tag first
layout with <table>	tables only for data; use flex / grid
missing alt on images	always add meaningful alt text

Mistake	Fix
forgetting ; in CSS	every declaration ends with ;
using id for many elements	ids must be unique - use class
z-index not working	element must be positioned (not static)
height on tables	let content decide the height
margin vs padding confusion	margin outside the border, padding inside
fixed widths on phones	use %, fr, max-width, media queries
external CSS not applied	check <link rel="stylesheet" href=...> path, in <head>

Practice tasks

- Create a page with headings, paragraphs, links, an image, lists, a table, media and a semantic layout (header, nav, main, footer).
- Make a registration form with text, email, password, radio, checkbox, select and a submit button; add required and a pattern.
- Style it with an external CSS file; predict which colour wins when p, .test , #demo and an inline style conflict.
- Build a navigation bar with Flexbox and a card gallery with Grid; make the cards one column below 700px.
- Add hover effects with transition + transform, a gradient button and a box-shadow card; define colours with CSS variables.

- **BIG IDEA** Learn by building: write the code, open it in the browser, change one thing, watch what changes. Use the browser's Inspect tool to see boxes, margins and which rules win.